
Understanding Goal Generalisation in Sequential Reinforcement Learning

Jason Ross Brown
University of Cambridge
Cambridge, United Kingdom
jrb239@cam.ac.uk

Edward James Young
University of Cambridge & Geodesic Research
Cambridge, United Kingdom

Abstract

Reinforcement learning agents often exhibit unintended goal-directed behaviour outside their training distribution, but we currently lack a principled understanding of how such agents will generalise to novel environments based on their training history. We address this gap for agents trained sequentially on one or more tasks. We study over 100 sequential training pipelines, evaluating behaviour across over 250 out-of-distribution environments. We find that salient features drive generalisation, and that goals learnt early in training can persist and influence those acquired later. To explain these phenomena, we introduce *latent policy gradients*, a method that predicts what out-of-distribution behaviour a training pipeline will likely induce. Our method simulates the evolution of low-dimensional latent variables during training according to what would achieve high reward on the training objective with respect to a simple model of how the latent variables map to behaviour. It achieves strong predictive accuracy, generalises to unseen types of training pipeline, and is interpretable. Our findings demonstrate that while out-of-distribution RL agent behaviour is dependent on the whole training pipeline, this dependence has an underlying structure we can capture, laying groundwork for understanding goal generalisation from a developmental perspective.

1 Introduction

Reinforcement learning (RL) agents trained in one environment may exhibit very different behaviour when placed in environments not seen in training [Kirk et al., 2023, Langosco et al., 2023]. While unpredictable out-of-distribution behaviour can be found across machine learning methods, RL can sometimes lead to agents which continue to show coherent, goal-directed behaviour out-of-distribution. A particular instance of this phenomenon is *goal misgeneralisation*, in which RL agents may learn to pursue a goal that obtains high reward within training environments, but not on out-of-distribution samples [Shah et al., 2022, Langosco et al., 2023]. As agents trained via RL are increasingly deployed in the real world, they will encounter situations outside those they are trained on; understanding and predicting behaviour in these situations in advance is therefore necessary for providing guarantees of safety and reliability [Hubinger et al., 2021, Hendrycks et al., 2023, Ngo et al., 2025].

This is particularly pressing in modern frontier AI systems, which are typically trained through a variety of stages including self-supervised pre-training, supervised fine-tuning, reinforcement learning from human feedback [Ouyang et al., 2022, Bai et al., 2022a, Rafailov et al., 2023], and reinforcement learning against verifiable rewards [DeepSeek-AI et al., 2025]. Some of these stages are explicitly intended to shape the values the system will act on, as in Constitutional AI [Bai et al., 2022b] and Character Training [Anthropic, 2024, Maiya et al., 2025]. Additionally, existing empirical work has surfaced unexpected and often concerning behaviours in frontier AI systems, particularly when models undergo further training on new objectives [Betley et al., 2025, Taylor et al., 2025,

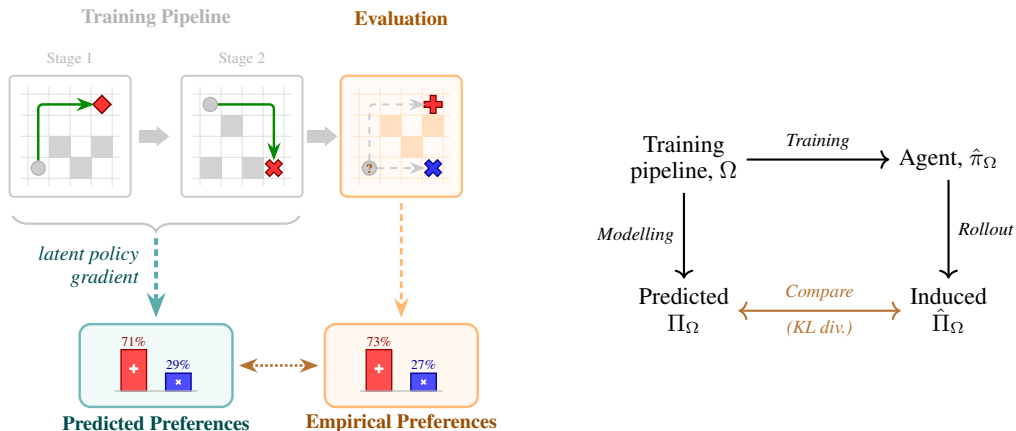


Figure 1: **Left: Illustration of our experimental design.** Our experimental design is covered in Section 3. RL agents are trained on pipelines involving either one or two stages (e.g., trained to pursue $\blacklozenge$ in stage 1 and $\times$ in stage 2). They are then evaluated in out-of-distribution environments containing two objects (e.g., $+$ and $\times$) to generate an empirical preference distribution. We explore these distributions in Section 4. Then, in Section 5, we introduce a method of predicting preference from training pipelines—the *latent policy gradients* method—and compare the fit between the predicted and empirical preferences. **Right: The goal generalisation modelling problem.** Training pipelines are used to train agents, which are then rolled out in evaluation environments to produce an induced preference function. Modelling gives predicted preference functions. We find the fit of our model by comparing the average KL-divergence between the predicted and induced preference functions.

MacDiarmid et al., 2025, Betley et al., 2026, Dubiński et al., 2026]. These findings illustrate *what* can go wrong with multi-stage training of AI systems deployed out-of-distribution, but we lack a predictive understanding of *how* these behaviours arise.

This paper seeks to advance our understanding of *goal generalisation* in reinforcement learning agents—given the training history of an agent and knowledge of its in-distribution behaviour, or that of similar agents, how can we predict the behaviour of that agent on out-of-distribution samples? We address this question specifically in the case of *transfer learning*, in which RL agents are trained on multiple tasks in sequence [Khetarpal et al., 2022, Iman et al., 2023]. Our hope is to understand this question at a fundamental level, building a principled theory of how training pipelines shape what agents learn to value. Building such an understanding requires solid theoretical foundations, which in turn call for controlled settings where training configurations can be systematically varied. We therefore explore the goal generalisation problem in a deliberately controlled toy setting.

We train RL agents within maze environments to pursue objects that have a particular colour and shape. Agents are trained either to pursue a single goal object, or sequentially to pursue two different goal objects across two training stages. We then evaluate these agents on out-of-distribution environments containing two objects, and measure the frequency at which they select one object over another. This setup is detailed in Section 3. In Section 4 we explore these preferences empirically, highlighting several important phenomena.

We subsequently attempt to understand and explain our observations in Section 5. Inspired by models of associative learning from behavioural psychology [Rescorla, 1972], we posit that agent goals can be understood in terms of a few *latent variables* which evolve during training in a predictable and understandable manner. We term this evolution *latent policy gradients*. We first show that our method provides a good fit to our empirical preference data, despite having only a few parameters. Then, we show that the hyperparameters of our method admit a natural interpretation in terms of feature similarity, and that the evolution of our latent variables can be understood as iterated projection onto hyperplanes.

Illustrated in Figure 1, the main contributions of our paper are as follows:

1. We provide an empirical analysis of goal generalisation behaviour of RL agents trained with over 100 different training pipelines, each evaluated on over 250 out-of-distribution environments. We demonstrate a number of empirical phenomena through both single case studies and population-level effects.
2. We provide a novel formulation of the goal generalisation problem as predicting the out-of-distribution behaviour of RL agents based on their training pipeline.
3. We provide a method for solving this problem—*latent policy gradients*—and show that it achieves good fit with our empirical data while having interpretable dynamics and hyperparameters.¹

2 Background

An RL agent exhibits *goal misgeneralisation* when it performs competently but pursues unintended goals on out-of-distribution (OOD) inputs [Shah et al., 2022, Langosco et al., 2023]—the agent behaves competently, but toward the wrong objective. Goal misgeneralisation poses a significant gap in our ability to safely deploy RL agents, as it can lead to agents that perform well during training and evaluation but behave unexpectedly once deployed. In this paper, we investigate the broader problem of *goal generalisation*: given knowledge of an agent’s training pipeline, can we predict how the agent’s learnt behaviour will generalise to unseen environments?

As discussed in Section 1, modern frontier AI systems are trained through multi-stage pipelines, some stages of which are explicitly designed to shape the system’s general behaviour across many environments. While transfer and continual learning have been studied extensively in general machine learning [Iman et al., 2023] and in reinforcement learning specifically [Khetarpal et al., 2022], comparatively little is known about how sequences of goal-directed RL training stages shape the final OOD behaviour of an agent. This is the regime our work targets. A broader discussion of related work is provided in Appendix A.

3 Experimental setting

3.1 The maze task

We train CNN-based RL agents on procedurally generated maze navigation tasks (see Appendix B.1 for details on the training algorithm and network architecture). Each episode, the agent (represented by a grey circle, ●) is placed in a procedurally generated 8×8 maze containing a goal object with a fixed colour and shape, *e.g.*, a red cross, ✕. Each episode of RL training occurs in a new maze. The agent’s observations are 128×128 RGB pixel images. See Appendix B.2, Figure 4 for maze generation details and an example maze.

Each turn, the agent can move up, down, left, or right, with movement into a wall cell resulting in no change. A reward of +1 is generated when the agent reaches the goal object, whereby the episode terminates; every other transition results in a reward of -0.1. The environment has a fixed horizon of 200 steps, after which the episode is terminated with no further reward. For some environments, we also include a distractor object of fixed shape and colour distinct from the goal. Distractor objects are visual and do not affect rewards. The colours of goal and distractor objects can vary between black, red, and blue, and shapes vary between crosses (✕), diamonds (◆), plusses (+), and rings (○), giving a total of 12 different possible objects. Visualisations of the shapes and colours are given in Appendix B.2, Figure 5.

3.2 Training pipelines and evaluation

We denote by Ω a specific sequence of goal-directed RL training stages, which we term a *training pipeline*, and by π_Ω the agent that results from training through that pipeline. To investigate how training pipelines affect goal generalisation, we collect data on the preferences of agents trained through a range of pipelines. We consider training pipelines consisting of either one or two stages. In each stage, the agent is trained for 3 million steps with PPO [Schulman et al., 2017] on mazes

¹Code and data for reproducing our experiments are available at <https://github.com/jr-brown/latent-policy-gradients>.

with a fixed goal object, and either one or no distractor objects. For example, a two-stage pipeline may consist of first training the agent on mazes with $\blacklozenge$ goal and no distractor, and then subsequently training the agent on mazes with $\times$ goal and $+$ distractor. See Appendix H.2, Figure 15, for example learning curves from our agents.

For each of the 12 shape and colour combinations we train an agent in the single stage training pipeline. We then fine-tune independent copies of each of these agents with all possible **red** and black goals to obtain two-stage pipeline results.² This gives a total of 108 agents³ for our analysis in Section 4.

For Section 5, we extend our dataset by training additional agents with distractor objects present during training. Distractors can have any colour and shape available to training goals, with the addition of **green**. This yields 190 unique training pipelines (both single and two stages, with and without distractors), giving a total of 298 agents. Full details of the distractor pipeline configurations are given in Appendix B.1.

For each agent, we define its preferences over objects operationally as follows. For each possible pair of objects, we rollout 100 episodes in mazes containing that pair, and record the fraction of times the agent reaches each of the two objects first, and the fraction of times the episode terminates at 200 steps without either object being reached. This includes objects with a novel colour (**green**) and two novel shapes ($\bullet$, $\diamond$) not seen in any training pipeline. This gives a total of 276 evaluation environments.⁴ We denote the resulting empirical distribution over outcomes for agent $\hat{\pi}_\Omega$ as its *empirical preference function*, $\hat{\Pi}_\Omega$. Our modelling problem in Section 5 is to predict $\hat{\Pi}_\Omega$ from Ω alone.

4 Empirical analysis of agent values

4.1 Method

Naively, one might expect agents to act erratically out of distribution, or if they were to act coherently, to simply pursue the last goal they were trained on. We investigate both of these assumptions. First, we test whether our trained agents display coherent OOD preferences at all. *A priori*, there is no reason to expect this: agents are trained to navigate toward a single goal object per environment, and their behaviour on environments containing novel object pairs is unconstrained by the training objective. Having investigated the coherence of these preferences, we then explore their structure, and how they relate to the agents’ training histories.

The notion of coherence we employ is *Boltzmann-rationality*: for each agent, each object is assigned a scalar score, and the probability of preferring object A to B is given by $\sigma(\text{score}(A) - \text{score}(B))$, where σ is the logistic function. This compresses the full set of $\binom{n}{2}$ pairwise preference probabilities into just n scores. Boltzmann-rationality is a stochastic analogue of the transitivity requirement of classical preference theory [Von Neumann and Morgenstern, 1947], where if A is preferred to B , and B preferred to C , then A must be preferred to C . Analogously, Boltzmann-rationality implies that if A is more likely to be chosen than B , and B more likely than C , then A must be more likely to be chosen than C .⁵ If an agent’s preferences frequently violate this condition, they cannot be well-described by a set of scores, and the model will be unable to predict held-out pairwise choices. We test the Boltzmann-rationality assumption by first dividing the data into $K = 4$ folds, and then for each fold fitting scores on the other folds using the Elo algorithm [Elo, 2008], and evaluating the fitted scores’ ability to predict pairwise preferences from the initial held-out fold. Specifically we compute the directional accuracy of the preferences predicted by the Boltzmann-rationality model, *i.e.*, the proportion of held-out pairs for which the model correctly predicts which object is selected more frequently. For more details on the fitting procedure, see Appendix C.1.

²Since our agents receive single-channel RGB observations, training with **blue** second-stage goals would be symmetric with **red** goals (each occupying one colour channel). We therefore omit **blue** second-stage goals to avoid unnecessary repeats.

³There are 3 colours and 4 shapes, resulting in 12 possible first-stage training goal objects. For second stage-training, omitting **blue** goal objects leaves 2 colours and 4 shapes, and so 8 possible second-stage training goal objects. This gives a total of 12 single-stage agents and $8 \times 12 = 96$ two-stage agents.

⁴There are 24 possible objects (4 colours and 6 shapes). There are then $\binom{24}{2} = 276$ evaluation environments.

⁵More precisely, Boltzmann-rationality requires $\text{logit}(\mathbb{P}(A \succ C)) = \text{logit}(\mathbb{P}(A \succ B)) + \text{logit}(\mathbb{P}(B \succ C))$.

4.2 Results

Out-of-distribution agent preferences are coherent. We find our Elo scores have 89.73% directional accuracy on held-out pairs in our 4-fold cross-validation ($\pm 0.18\%$ SE; see Appendix F.1). The high validation accuracy confirms that agent preferences over OOD goals are approximately coherent in the sense described above—a non-trivial finding given that nothing in the training objective constrains OOD preferences in this manner.

Having validated the Elo scores as a good summary of agent preferences, we can now use them to investigate the structure of these preferences, and how they relate to training history. To summarise how much each agent values individual features, we average Elo scores over all objects containing a given feature to produce *marginalised Elo scores*. We observe three clear patterns in the structure of these preferences, which we summarise here. Full details, case studies, population-level analyses, and all supporting figures are given in Appendix H.1. In this section we only analyse training pipelines without distractors; Appendix H.3 contains breakdowns of all marginalised Elo scores across all agents.

Some features are more salient in learning than others. Across our single-stage pipelines, we find that when agents are evaluated OOD, they tend to pursue objects sharing specific features of their training goal more than others. In agreement with prior work on the inductive biases of CNNs [Ritter et al., 2017, Feinman and Lake, 2018], shape tends to drive generalisation more strongly than colour, with **X**-shape being the most salient feature and black the least (Figure 9). We also observe *feature confusion*, where training on one feature can lead to valuing or de-valuing another (Figure 10). We will return to this phenomenon in Section 5.

Values persist across training stages. In two-stage training pipelines, values learnt for features in the first stage’s goal persist after the second stage, even when those features are not part of the second training objective (Figure 11). This holds both whether the two goals share some other feature or not. Additionally, agents trained on more unique features generalise to pursue more objects (Figure 12).

Repeated goal features’ values are strengthened, and inhibit new values forming. When a feature is shared between two training stages, that feature’s value is strengthened relative to if it only appeared in the second stage. Conversely, the non-shared feature in the second-stage goal is valued much less than it would be if the shared feature was not present, indicating that existing values for features of the training goal inhibit the formation of new values (Figure 13). These findings are consistent with known plasticity effects in continual learning [Nikishin et al., 2022, 2023, Dohare et al., 2024] and gradient starvation [Pezeshki et al., 2021], whereby dominant features suppress learning of others. As a corollary, we observe strong ordering effects when goals share a feature: flipping the order of training stages can significantly change the agent’s final values (Figure 14).

5 Understanding generalisation through latent policy gradients

In this section, we attempt to understand the generalisation effects observed in Section 4.2 by modelling agent preferences as arising from low-dimensional latent variables that evolve during training via policy gradient ascent.⁶

5.1 Problem Formalism

We now provide additional formal detail on the preference function $\hat{\Pi}_\Omega$ introduced in Section 3.2. In the set of objects we include the null-object, $\mathbf{0}$, which corresponds to an episode terminating without an object being reached. We write $\hat{\Pi}_\Omega(\phi^{(a)} | \phi^{(a)}, \phi^{(b)}, \mathbf{0})$ to denote the empirical probability that when the agent $\hat{\pi}_\Omega$ is placed in an environment containing objects with features $\phi^{(a)}, \phi^{(b)}$, it terminates the episode by reaching the object with features $\phi^{(a)}$. Further, $\hat{\Pi}_\Omega(\mathbf{0} | \phi^{(a)}, \phi^{(b)}, \mathbf{0})$ is used to denote the empirical probability that the episode will terminate at some fixed horizon length without any object being reached (in our case, 200 steps).

⁶Prior work in Inverse RL and Game Theory indicate preferences capture the underlying motivations of an agent [Von Neumann and Morgenstern, 1947, Skalse et al., 2023], we are also partially inspired by models of associative learning from behavioural psychology [Rescorla, 1972].

The modelling problem can then be formulated as follows: find a mapping from training pipelines to preference functions, $\Omega \mapsto \Pi_\Omega$, which minimises the KL-divergence from the empirically observed preference function. This is illustrated in Figure 1. In other words, we wish to minimise the following *modelling loss*:

$$D\left(\hat{\Pi}_\Omega\left(\bullet \middle| \phi^{(a)}, \phi^{(b)}, \mathbf{0}\right) \middle| \middle| \Pi_\Omega\left(\bullet \middle| \phi^{(a)}, \phi^{(b)}, \mathbf{0}\right)\right), \quad (1)$$

which we average over $\Omega \sim \mathcal{D}_{\text{train}}, (\phi^{(a)}, \phi^{(b)}) \sim \mathcal{D}_{\text{eval}}$.

5.2 The latent policy gradient method

There are many ways to specify a mapping from training pipelines to predicted preference functions, $\Omega \rightarrow \Pi_\Omega$. In this section, we introduce the core of our modelling approach—the *latent policy gradients* method.

We hypothesise that the preferences of policies can be encapsulated by a small collection of *latent variables* which determine agent behaviour, $\mathbf{w}$. Trivially, a large enough collection of latents can capture the agent’s behaviour—indeed, we could take $\mathbf{w}$ to just be the entire parameter set of our agent’s network. Our goal therefore is to find a set of *low-dimensional* latents, which is much smaller than the number of parameters in the network, and which have readily interpretable meaning.

The first step of the latent policy gradients method is to specify a parametrisation of preference functions according to latent variables, $\Pi(\bullet \middle| \bullet; \mathbf{w})$. This parametrisation may include hyperparameters. Given this parametrisation, we must then specify how the latent parameters, $\mathbf{w}$, depend on the training pipeline, Ω . In other words, we must specify a mapping $\Omega \mapsto \mathbf{w}_\Omega$. Ideally, this map itself is again interpretable and meaningful. With some slight abuse of notation, we will then use $\Pi(\bullet \middle| \bullet; \mathbf{w}_\Omega)$ as our predicted preference function, $\Pi_\Omega(\bullet \middle| \bullet)$.

For each training pipeline, Ω , we specify $\mathbf{w}_\Omega$ by performing gradient ascent for a fixed number of steps on modified policy objectives *over preference functions*. We also include entropy regularisation in our objective [Haarnoja et al., 2017, 2019]. Entropy regularisation allows us to model how effectively we expect the agents to pursue their training objectives via a hyperparameter, τ . Concretely, when training on an Ω with a single stage with a single goal, $\phi^{(g)}$, we use the following policy objective over preference functions:

$$J(\Pi) = \Pi\left(\phi^{(g)} \middle| \phi^{(g)}, \mathbf{0}\right) + \tau h\left(\Pi\left(\phi^{(g)} \middle| \phi^{(g)}, \mathbf{0}\right)\right) \quad (2)$$

where $h(\bullet)$ is the binary entropy function⁷. When the training environment of the stage includes a distractor object with features $\phi^{(d)}$, we include that distractor’s features in the conditioning set, *i.e.*, replace $\Pi\left(\phi^{(g)} \middle| \phi^{(g)}, \mathbf{0}\right)$ with $\Pi\left(\phi^{(g)} \middle| \phi^{(g)}, \phi^{(d)}, \mathbf{0}\right)$ in Equation (2). When the training pipeline contains multiple stages, we fit on corresponding J s in sequence, with the final $\mathbf{w}$ of one gradient ascent initialising the $\mathbf{w}$ of the next.

5.3 Preference function parametrisation

To apply latent policy gradients to our problem, we need to specify a parametrisation of our preference functions, Π , in terms of latent variables, $\mathbf{w} \mapsto \Pi(\bullet \middle| \bullet; \mathbf{w})$. In Section 4.2 we showed that agent preferences are well-described by Elo scores, which assume Boltzmann-rational preferences. We therefore adopt the same assumption here [Bradley and Terry, 1952, Wulfmeier et al., 2016, Christiano et al., 2017, Skalse and Abate, 2023], and additionally parametrise values as linear in both goal features and the latent variables. Specifically, for a goal with features $\phi^{(g)}$, the preference function is:

$$\Pi\left(\phi^{(g)} \middle| \phi^{(g)}, \mathbf{0}; \mathbf{w}\right) = \frac{\exp\left(\phi^{(g)} \cdot S\mathbf{w}\right)}{1 + \exp\left(\phi^{(g)} \cdot S\mathbf{w}\right)}. \quad (3)$$

We choose the dimension of $\mathbf{w}$ to be 10, equal to the total number of object features.⁸ In Appendix E we show that performance saturates at the number of feature dimensions, as expected, and can be reduced to 8 without degradation. Note that this is much smaller than the 4.8 million parameters of our trained CNN-based agents. The saliency matrix, S , is a hyperparameter that controls which

⁷ $h(p) = -p \log(p) - (1-p) \log(1-p)$ is the entropy of a Bernoulli random variable with probability p

⁸Note that this parametrisation is equivalent to modelling Elo scores as linear in goal features, with $V(\phi^{(g)}) = \phi^{(g)} \cdot S\mathbf{w}$.

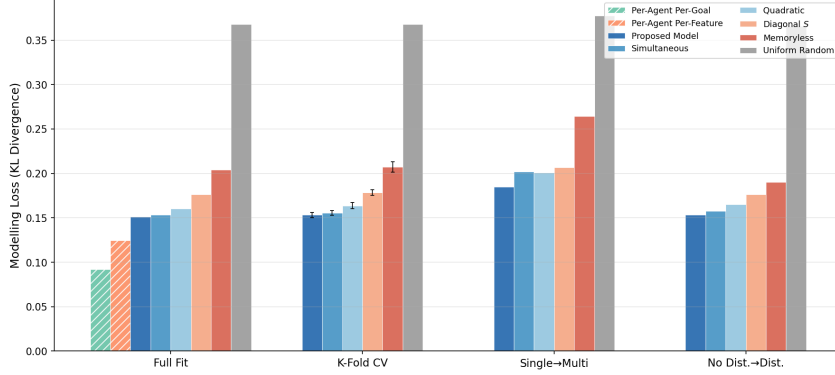


Figure 2: **Model comparison.** Average modelling loss (Equation (1)) across four evaluations (described in main text). Error bars show standard error for K-fold CV. Two per-agent lower bounds (Full Fit only) are also shown. Lower is better. Exact values are given in Table 5.

features are learnt to be valued faster, as well as modelling feature confusion. As discussed in Section 4.2, we observe empirically that training on one set of features can cause agents to pursue goals with features they were not trained on; the off-diagonal entries of S capture these cross-feature effects (see Appendix H.1, Figure 10). We constrain it to be upper-triangular to account for rotational invariance in our model. We also learn an initial value $w_0 \in \mathbb{R}$, so that w is initialised to $w_0 \mathbf{1}$ before simulating each training pipeline. Like τ , S and w_0 are fixed while performing gradient ascent on the policy objective over preference functions, Equation (2), but optimised in an outer loop to reduce the modelling loss, Equation (1). For details on our full algorithm including the hyperparameter fitting procedure, see Appendix C.

5.4 Results

To analyse the suitability of our model, we test it along with four alterations, a baseline, and two lower bounds across four different evaluations. In each evaluation we fit the hyperparameters of the model to a subset of preference data, and then compute the modelling loss (Equation (1)) across some evaluation subset.

We compare against four alternatives: **Diagonal S** , which restricts the saliency matrix to be diagonal (no feature confusion); **Memoryless**, which fits only on the last training objective; **Simultaneous**, which ignores the ordering of training stages; and **Quadratic**, which expands the feature space to include pairwise interactions (with diagonal S to limit parameters).⁹ As a baseline, we consider the **uniform random** predictor, which predicts equal probability for each outcome (goal a , goal b , or neither). We also report two loss lower bounds which fit values directly to each agent’s observed preferences rather than predicting them from training pipelines. The **per-agent per-feature** bound fits a value for each (agent, feature) combination and computes goal values as linear combinations, giving the lowest achievable loss for any method which assumes goal values decompose linearly over features—this includes our proposed model and all alternatives except Quadratic. The **per-agent per-goal** bound fits an independent value for each (agent, goal) pair—equivalent to fitting Elo scores independently per agent—giving the lowest achievable loss for any Boltzmann-rational model. Since these bounds fit values to observed preferences rather than predicting them from training pipeline descriptions, they are only applicable to the Full Fit evaluation and require substantially more parameters ($n_{\text{agents}} \times n_{\text{goals}}$ and $n_{\text{agents}} \times n_{\text{features}}$ respectively). By contrast, our proposed model fits a single shared set of 57 hyperparameters across all 298 agents.

The four evaluations are: fitting the hyperparameters on the entire dataset and reporting the overall fit; performing 4-fold cross-validation (holding out 25% of training pipelines for evaluation); fitting hyperparameters on just the single-stage pipelines and reporting model fit on the two-stage pipelines; fitting hyperparameters on pipelines with no distractors and reporting model fit on the pipelines

⁹The expanded feature space contains n base features, n self-interaction terms (ϕ_i^2), and $\binom{n}{2}$ unique cross-interaction terms ($\phi_i \phi_j = \phi_j \phi_i$ for $i < j$). Since each symmetric pair shares identical learning dynamics, the effective number of saliency parameters is $2n + \binom{n}{2} = 65$ for $n = 10$, giving 67 total hyperparameters.

containing distractors. These last two evaluations investigate how robust a model is when used to predict the effects of training pipelines that are qualitatively different from those used to fit its hyperparameters.

Our results are given in Figure 2. Our proposed model achieves the lowest modelling loss across all evaluations. Notably, a model fit only on single-stage pipelines successfully predicts the preferences of agents trained on two-stage pipelines, and a model fit without distractors generalises to pipelines containing them. We also see from the alternatives that our model captures underlying patterns in the data better than one would be able to making the assumptions corresponding to each alternative, *e.g.*, training is clearly not memoryless. We do note that in contrast to the empirical evidence that ordering effects exist (Appendix H.1, Figure 14), the simultaneous alternative—which explicitly ignores training order—performs close to our proposed model. This implies that while ordering effects are real, they account for a relatively small fraction of the total variation in preferences, and the specific form of our model perhaps does not yet capture their nature fully.

We additionally validate the fits using total variation distance, Brier score, and directional accuracy, finding that the model ranking is consistent across all metrics (Appendix F). We give our fitted hyperparameters for the full dataset in Appendix D, along with a brief interpretation of them.

5.5 Interpretation of latent policy gradients

As shown in the previous section, the latent policy gradients method provides a good fit to the data. However, we also claim that it is interpretable. In this section, we substantiate this claim in two ways. Firstly, we demonstrate that the latent policy gradients dynamics correspond to a sequence of projection operations. Secondly, we show that the hyperparameters learned by our method—specifically the saliency matrix S —induce a similarity metric between features.

Multi-stage training is iterated projection. For the linear Boltzmann-rational preference function, Equation (3), the gradient of the modified policy objective Equation (2) can be computed analytically. In Appendix G.1, we show that, in the case of no distractor object:

$$\nabla_{\omega} J = (1 - \tau v^g) \sigma(v^g) (1 - \sigma(v^g)) S^T \phi^{(g)}, \quad (4)$$

where $v^g = \phi^{(g)} \cdot S\mathbf{w}$ is the value assigned to the goal object. This implies that the latents are exclusively updated in the direction $S^T \phi^{(g)}$. Although we perform only a finite number of gradient updates, we can use Equation (4) to approximate the final weights by solving $\nabla_{\omega} J = 0$. This gives $v^g = \phi^{(g)} \cdot S\mathbf{w} = \tau^{-1}$. Therefore (see Appendix G.2), if we begin with weights $\mathbf{w}_0$, the updated weights are

$$\mathbf{w} = \mathbf{w}_0 + \left(\frac{\tau^{-1} - \phi^{(g)} \cdot S\mathbf{w}_0}{\|S^T \phi^{(g)}\|_2^2} \right) S^T \phi^{(g)}. \quad (5)$$

Consequently, multi-stage training can be viewed as iteratively projecting the weights onto the hyperplanes $\phi^{(g)} \cdot S\mathbf{w} = \tau^{-1}$. We visualise this in Figure 3.

The latent saliency matrix induces a natural similarity metric over feature space. Given Equation (5), we can ask how training on a goal $\phi^{(g)}$ effects the *value* that the model assigns to another object, ϕ' , $\phi' \cdot S\mathbf{w}$. Note from Equation (3) that these values determine the preferences of the policy. If $\mathbf{w}_0 = 0$, the value assigned to ϕ' is

$$\tau^{-1} \phi' \cdot SS^T \phi^{(g)} / \|S^T \phi^{(g)}\|_2^2. \quad (6)$$

Equation (6) tells us that training on one goal increases the value of other objects in accordance with their similarity with the goal, as measured by their inner product with respect to the matrix SS^T . We can therefore interpret SS^T as an *object similarity metric*, as judged by the policy network architecture. In particular, we can interpret the diagonal elements of SS^T as feature-specific learning rates—the relative strengths of each feature in determining generalisation.

6 Discussion

This paper formulates the goal generalisation problem as predicting OOD behaviour from training pipelines, and introduces latent policy gradients as a solution. The best-performing parametrisation

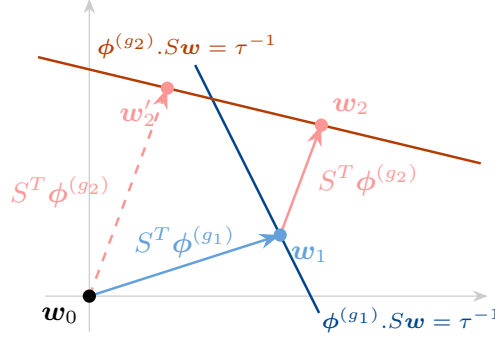


Figure 3: **Multi-stage training is iterated projection.** Latent policy gradient shifts the latent variables w in the $S^T \phi^{(g)}$ direction until they intersect with the hyperplane $\phi^{(g)} \cdot S w = \tau^{-1}$. The result of training (to convergence) first on $\phi^{(g_1)}$, and then on $\phi^{(g_2)}$ is shown by w_2 . The result of training to convergence on $\phi^{(g_2)}$ alone is shown by w'_2 .

uses a saliency matrix to capture differing feature learning rates and cross-feature confusion—phenomena observed in our empirical analysis. Beyond empirical fit, latent policy gradients admit interpretation: their dynamics correspond to iterated projection onto hyperplanes, and their hyperparameters define a feature similarity metric. Unlike standard explainability methods, which characterise what a given model has learned, latent policy gradients predict how preferences form during training.

Neural Network Plasticity. Two of the phenomena we reported—value persistence and value inhibition—are consistent with plasticity effects in continual learning [Nikishin et al., 2022, 2023, Dohare et al., 2024, Lyle et al., 2024], gradient starvation [Pezeshki et al., 2021], and the emergence of dormant neurons [Sokar et al., 2023]. We show the impact these effects have on goal generalisation in multi-stage goal-directed RL, and further show that they can be captured by a simple latent model with interpretable structure.

Limitations and future work. Our experiments used a single architecture and algorithm (PPO with a CNN) on a single domain (maze navigation) with one- or two-stage pipelines; extending to other architectures and algorithms (particularly action-value methods such as DQN [Mnih et al., 2013]), domains, and longer pipelines is important future work. Our saliency findings (*e.g.*, shape over colour) reflect CNN inductive biases, though the LPG framework itself is architecture-agnostic. Our agents were each trained with a single random seed, though the large number of pipeline permutations provides symmetries that serve a similar role. The preference function parametrisation we used assumes that goal values decompose linearly over features. This is well-suited to the factorial structure of our domain, but may not hold in settings where goals have more complex, non-compositional structure. The framework accommodates alternative parametrisations, but we have not yet explored them. Finally, our predictions of agent preferences were not perfect, and we welcome future work which improves performance on the goal generalisation problem on our dataset.

Broader impacts. This work is motivated by AI safety: predicting how training pipelines shape OOD behaviour is necessary for safe deployment of RL systems. By providing interpretable methods for predicting goal generalisation, we aim to contribute to building more understandable AI systems. We do not foresee negative societal consequences from this foundational research.

Conclusion. The values of RL agents are path dependent, and prior training can significantly influence how a model generalises OOD. However, this path-dependence has underlying structure. Latent policy gradients provide a means of predicting goal generalisation via the evolution of a small number of latent variables. Our ultimate motivation is understanding how multi-stage training pipelines shape the values of frontier AI systems, and we see the problem formulation, evaluation methodology, and baseline method established here as a foundation for this goal. The mathematical structure of latent policy gradients is general, and we believe it provides a strong foundation for extending this work to more complex systems.

References

- Pieter Abbeel and Andrew Y. Ng. Apprenticeship learning via inverse reinforcement learning. In *Twenty-first international conference on Machine learning - ICML '04*, page 1, Banff, Alberta, Canada, 2004. ACM Press. doi: 10.1145/1015330.1015430. URL <http://portal.acm.org/citation.cfm?doid=1015330.1015430>.
- Stephen Adams, Tyler Cody, and Peter A. Beling. A survey of inverse reinforcement learning. *Artificial Intelligence Review*, 55(6):4307–4346, August 2022. ISSN 0269-2821, 1573-7462. doi: 10.1007/s10462-021-10108-x. URL <https://link.springer.com/10.1007/s10462-021-10108-x>.
- Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete Problems in AI Safety, July 2016. URL <http://arxiv.org/abs/1606.06565>. arXiv:1606.06565 [cs].
- Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Eric Winsor, Jerome Wynne, Yarin Gal, and Xander Davies. AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents, April 2025. URL <http://arxiv.org/abs/2410.09024>. arXiv:2410.09024 [cs].
- Anthropic. Claude’s Character, August 2024. URL <https://www.anthropic.com/research/claude-character>.
- Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, Nicholas Joseph, Saurav Kadavath, Jackson Kernion, Tom Conerly, Sheer El-Showk, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Tristan Hume, Scott Johnston, Shauna Kravec, Liane Lovitt, Neel Nanda, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, Ben Mann, and Jared Kaplan. Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback, April 2022a. URL <http://arxiv.org/abs/2204.05862>. arXiv:2204.05862 [cs].
- Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosuite, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemi Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. Constitutional AI: Harmlessness from AI Feedback, December 2022b. URL <http://arxiv.org/abs/2212.08073>. arXiv:2212.08073 [cs].
- Leonard Bereska and Efstratios Gavves. Mechanistic Interpretability for AI Safety – A Review, August 2024. URL <http://arxiv.org/abs/2404.14082>. arXiv:2404.14082 [cs].
- Jan Betley, Jorio Cocola, Dylan Feng, James Chua, Andy Ardit, Anna Szyber-Betley, and Owain Evans. Weird generalization and inductive backdoors: New ways to corrupt llms. *arXiv preprint arXiv:2512.09742*, 2025.
- Jan Betley, Daniel Tan, Niels Warncke, Anna Szyber-Betley, Xuchan Bao, Martín Soto, Nathan Labenz, and Owain Evans. Emergent Misalignment: Narrow finetuning can produce broadly misaligned LLMs. *Nature*, 649(8097):584–589, January 2026. ISSN 0028-0836, 1476-4687. doi: 10.1038/s41586-025-09937-5. URL <http://arxiv.org/abs/2502.17424>. arXiv:2502.17424 [cs].
- Ralph Allan Bradley and Milton E. Terry. Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons. *Biometrika*, 39(3/4):324, December 1952. ISSN 00063444. doi: 10.2307/2334029. URL <https://www.jstor.org/stable/2334029?origin=crossref>.
- Jason R. Brown, Carl Henrik Ek, and Robert D. Mullins. Learning from Preferences and Mixed Demonstrations in General Settings, August 2025. URL <http://arxiv.org/abs/2508.14027>. arXiv:2508.14027 [cs].

- Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep Reinforcement Learning from Human Preferences. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/d5e2c0adad503c91f91df240d0cd4e49-Paper.pdf.
- Karl Cobbe, Oleg Klimov, Chris Hesse, Taehoon Kim, and John Schulman. Quantifying Generalization in Reinforcement Learning, July 2019. URL <http://arxiv.org/abs/1812.02341>. arXiv:1812.02341 [cs].
- Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging Procedural Generation to Benchmark Reinforcement Learning, July 2020. URL <http://arxiv.org/abs/1912.01588>. arXiv:1912.01588 [cs].
- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jia Shi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojuan Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanbiao Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *Nature*, 645(8081): 633–638, September 2025. ISSN 0028-0836, 1476-4687. doi: 10.1038/s41586-025-09422-z. URL <http://arxiv.org/abs/2501.12948>. arXiv:2501.12948 [cs].
- Shibhansh Dohare, J Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A Rupam Mahmood, and Richard S Sutton. Loss of plasticity in deep continual learning. *Nature*, 632(8026): 768–774, 2024. ISSN 0028-0836.
- Jan Dubiński, Jan Betley, Anna Szyber-Betley, Daniel Tan, and Owain Evans. Conditional misalignment: common interventions can hide emergent misalignment behind contextual triggers, 2026. URL <https://arxiv.org/abs/2604.25891>. _eprint: 2604.25891.
- Arpad E. Elo. *The rating of chessplayers, past and present*. Ishi Press International, Bronx, NY, 2. print edition, 2008. ISBN 978-0-923891-27-5.
- Reuben Feinman and Brenden M. Lake. Learning Inductive Biases with Simple Neural Networks, June 2018. URL <http://arxiv.org/abs/1802.02745>. arXiv:1802.02745 [cs].
- Roya Firoozi, Johnathan Tucker, Stephen Tian, Anirudha Majumdar, Jiankai Sun, Weiyu Liu, Yuke Zhu, Shuran Song, Ashish Kapoor, Karol Hausman, Brian Ichter, Danny Driess, Jiajun Wu, Cewu Lu, and Mac Schwager. Foundation models in robotics: Applications, challenges, and the future.

- The International Journal of Robotics Research*, 44(5):701–739, April 2025. ISSN 0278-3649, 1741-3176. doi: 10.1177/02783649241281508. URL <https://journals.sagepub.com/doi/10.1177/02783649241281508>.
- Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, November 2020. ISSN 2522-5839. doi: 10.1038/s42256-020-00257-z. URL <https://www.nature.com/articles/s42256-020-00257-z>.
- Melody Y. Guan, Manas Joglekar, Eric Wallace, Saachi Jain, Boaz Barak, Alec Helyar, Rachel Dias, Andrea Vallone, Hongyu Ren, Jason Wei, Hyung Won Chung, Sam Toyer, Johannes Heidecke, Alex Beutel, and Amelia Glaese. Deliberative Alignment: Reasoning Enables Safer Language Models, January 2025. URL <http://arxiv.org/abs/2412.16339>. arXiv:2412.16339 [cs].
- Pim de Haan, Dinesh Jayaraman, and Sergey Levine. Causal Confusion in Imitation Learning, November 2019. URL <http://arxiv.org/abs/1905.11979>. arXiv:1905.11979 [cs].
- Tuomas Haarnoja, Haoran Tang, Pieter Abbeel, and Sergey Levine. Reinforcement Learning with Deep Energy-Based Policies, July 2017. URL <http://arxiv.org/abs/1702.08165>. arXiv:1702.08165 [cs].
- Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, and Sergey Levine. Soft Actor-Critic Algorithms and Applications, January 2019. URL <http://arxiv.org/abs/1812.05905>. arXiv:1812.05905 [cs].
- Dylan Hadfield-Menell, Stuart J Russell, Pieter Abbeel, and Anca Dragan. Cooperative Inverse Reinforcement Learning. In D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016. URL https://proceedings.neurips.cc/paper_files/paper/2016/file/c3395dd46c34fa7fd8d729d8cf88b7a8-Paper.pdf.
- Dan Hendrycks, Mantas Mazeika, and Thomas Woodside. An Overview of Catastrophic AI Risks, October 2023. URL <http://arxiv.org/abs/2306.12001>. arXiv:2306.12001 [cs].
- Evan Hubinger, Chris van Merwijk, Vladimir Mikulik, Joar Skalse, and Scott Garrabrant. Risks from Learned Optimization in Advanced Machine Learning Systems, December 2021. URL <http://arxiv.org/abs/1906.01820>. arXiv:1906.01820 [cs].
- Mohammadreza Iman, Hamid Reza Arabnia, and Khaled Rasheed. A Review of Deep Transfer Learning and Recent Advancements. *Technologies*, 11(2):40, March 2023. ISSN 2227-7080. doi: 10.3390/technologies11020040. URL <https://www.mdpi.com/2227-7080/11/2/40>.
- Khimya Khetarpal, Matthew Riemer, Irina Rish, and Doina Precup. Towards Continual Reinforcement Learning: A Review and Perspectives, November 2022. URL <http://arxiv.org/abs/2012.13490>. arXiv:2012.13490 [cs].
- Diederik P Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Robert Kirk, Amy Zhang, Edward Grefenstette, and Tim Rocktäschel. A Survey of Zero-shot Generalisation in Deep Reinforcement Learning. *Journal of Artificial Intelligence Research*, 76: 201–264, January 2023. ISSN 1076-9757. doi: 10.1613/jair.1.14174. URL <http://jair.org/index.php/jair/article/view/14174>.
- Lauro Langosco, Jack Koch, Lee Sharkey, Jacob Pfau, Laurent Orseau, and David Krueger. Goal Misgeneralization in Deep Reinforcement Learning, January 2023. URL <http://arxiv.org/abs/2105.14111>. arXiv:2105.14111 [cs].
- Clare Lyle, Zeyu Zheng, Khimya Khetarpal, Hado van Hasselt, Razvan Pascanu, James Martens, and Will Dabney. Disentangling the Causes of Plasticity Loss in Neural Networks, February 2024. URL <http://arxiv.org/abs/2402.18762>. arXiv:2402.18762 [cs].

- Aengus Lynch, Benjamin Wright, Caleb Larson, Stuart J. Ritchie, Soren Mindermann, Evan Hubinger, Ethan Perez, and Kevin Troy. Agentic Misalignment: How LLMs Could Be Insider Threats, October 2025. URL <http://arxiv.org/abs/2510.05179>. arXiv:2510.05179 [cs].
- Monte MacDiarmid, Benjamin Wright, Jonathan Uesato, Joe Benton, Jon Kutasov, Sara Price, Naia Bouscal, Sam Bowman, Trenton Bricken, Alex Cloud, Carson Denison, Johannes Gasteiger, Ryan Greenblatt, Jan Leike, Jack Lindsey, Vlad Mikulik, Ethan Perez, Alex Rodrigues, Drake Thomas, Albert Webson, Daniel Ziegler, and Evan Hubinger. Natural Emergent Misalignment from Reward Hacking in Production RL, 2025. URL <https://arxiv.org/abs/2511.18397>. _eprint: 2511.18397.
- Sharan Maiya, Henning Bartsch, Nathan Lambert, and Evan Hubinger. Open Character Training: Shaping the Persona of AI Assistants through Constitutional AI, November 2025. URL <http://arxiv.org/abs/2511.01689>. arXiv:2511.01689 [cs].
- Mantas Mazeika, Xuwang Yin, Rishub Tamirisa, Jaehyuk Lim, Bruce W. Lee, Richard Ren, Long Phan, Norman Mu, Adam Khoja, Oliver Zhang, and Dan Hendrycks. Utility Engineering: Analyzing and Controlling Emergent Value Systems in AIs, February 2025. URL <http://arxiv.org/abs/2502.08640>. arXiv:2502.08640 [cs].
- IPL McLaren and NJ Mackintosh. Associative learning and elemental representation: II. Generalization and discrimination. *Animal learning & behavior*, 30(3):177–200, 2002. ISSN 0090-4996.
- Ulissee Mini, Peli Grietzer, Mrinank Sharma, Austin Meek, Monte MacDiarmid, and Alexander Matt Turner. Understanding and Controlling a Maze-Solving Policy Network, October 2023. URL <http://arxiv.org/abs/2310.08043>. arXiv:2310.08043 [cs].
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing Atari with Deep Reinforcement Learning, December 2013. URL <http://arxiv.org/abs/1312.5602>. arXiv:1312.5602 [cs].
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemaire, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, February 2015. ISSN 0028-0836, 1476-4687. doi: 10.1038/nature14236. URL <https://www.nature.com/articles/nature14236>.
- Akshat Naik, Patrick Quinn, Guillermo Bosch, Emma Gouné, Francisco Javier Campos Zabala, Jason Ross Brown, and Edward James Young. AgentMisalignment: Measuring the Propensity for Misaligned Behaviour in LLM-Based Agents, October 2025. URL <http://arxiv.org/abs/2506.04018>. arXiv:2506.04018 [cs].
- Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. Deep double descent: where bigger models and more data hurt*. *Journal of Statistical Mechanics: Theory and Experiment*, 2021(12):124003, December 2021. ISSN 1742-5468. doi: 10.1088/1742-5468/ac3a74. URL <https://iopscience.iop.org/article/10.1088/1742-5468/ac3a74>.
- Richard Ngo, Lawrence Chan, and Sören Mindermann. The Alignment Problem from a Deep Learning Perspective, May 2025. URL <http://arxiv.org/abs/2209.00626>. arXiv:2209.00626 [cs].
- Evgenii Nikishin, Max Schwarzer, Pierluca D’Oro, Pierre-Luc Bacon, and Aaron Courville. The Primacy Bias in Deep Reinforcement Learning, May 2022. URL <http://arxiv.org/abs/2205.07802>. arXiv:2205.07802 [cs].
- Evgenii Nikishin, Junhyuk Oh, Georg Ostrovski, Clare Lyle, Razvan Pascanu, Will Dabney, and André Barreto. Deep reinforcement learning with plasticity injection. *Advances in Neural Information Processing Systems*, 36:37142–37159, 2023.
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, March 2022. URL <http://arxiv.org/abs/2203.02155>. arXiv:2203.02155 [cs].

- John M Pearce. A model for stimulus generalization in Pavlovian conditioning. *Psychological review*, 94(1):61, 1987. ISSN 1939-1471.
- Mohammad Pezeshki, Oumar Kaba, Yoshua Bengio, Aaron C. Courville, Doina Precup, and Guillaume Lajoie. Gradient starvation: A learning proclivity in neural networks. *Advances in Neural Information Processing Systems*, 34:1256–1272, 2021.
- Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets, January 2022. URL <http://arxiv.org/abs/2201.02177>. arXiv:2201.02177 [cs].
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 53728–53741. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/a85b405ed65c6477a4fe8302b5e06ce7-Paper-Conference.pdf.
- Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dornmann. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. URL <http://jmlr.org/papers/v22/20-1364.html>.
- Deepak Ramachandran and Eyal Amir. Bayesian Inverse Reinforcement Learning.
- Robert A Rescorla. A theory of Pavlovian conditioning: Variations in the effectiveness of reinforcement and non-reinforcement. *Classical conditioning, Current research and theory*, 2:64–69, 1972.
- Samuel Ritter, David GT Barrett, Adam Santoro, and Matt M. Botvinick. Cognitive psychology for deep neural networks: A shape bias case study. In *International conference on machine learning*, pages 2940–2949. PMLR, 2017. ISBN 2640-3498.
- Andrei A. Rusu, Neil C. Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive Neural Networks, October 2022. URL <http://arxiv.org/abs/1606.04671>. arXiv:1606.04671 [cs].
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization Algorithms, August 2017. URL <http://arxiv.org/abs/1707.06347>. arXiv:1707.06347 [cs].
- Rohin Shah, Vikrant Varma, Ramana Kumar, Mary Phuong, Victoria Krakovna, Jonathan Uesato, and Zac Kenton. Goal misgeneralization: Why correct specifications aren’t enough for correct goals. *arXiv preprint arXiv:2210.01790*, 2022.
- Roger N Shepard. Toward a universal law of generalization for psychological science. *Science*, 237(4820):1317–1323, 1987. ISSN 0036-8075.
- Joar Skalse and Alessandro Abate. Misspecification in Inverse Reinforcement Learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(12):15136–15143, June 2023. ISSN 2374-3468, 2159-5399. doi: 10.1609/aaai.v37i12.26766. URL <https://ojs.aaai.org/index.php/AAAI/article/view/26766>.
- Joar Max Viktor Skalse, Matthew Farrugia-Roberts, Stuart Russell, Alessandro Abate, and Adam Gleave. Invariance in policy optimisation and partial identifiability in reward learning. In *International Conference on Machine Learning*, pages 32033–32058. PMLR, 2023. ISBN 2640-3498.
- Ghada Sokar, Rishabh Agarwal, Pablo Samuel Castro, and Utku Evci. The Dormant Neuron Phenomenon in Deep Reinforcement Learning, June 2023. URL <http://arxiv.org/abs/2302.12902>. arXiv:2302.12902 [cs].
- Mia Taylor, James Chua, Jan Betley, Johannes Treutlein, and Owain Evans. School of Reward Hacks: Hacking harmless tasks generalizes to misaligned behavior in LLMs, August 2025. URL <http://arxiv.org/abs/2508.17511>. arXiv:2508.17511 [cs].

- Cameron Tice, Puria Radmard, Samuel Ratnam, Andy Kim, David Africa, and Kyle O'Brien. Alignment Pretraining: AI Discourse Causes Self-Fulfilling (Mis)alignment, January 2026. URL <http://arxiv.org/abs/2601.10160>. arXiv:2601.10160 [cs].
- John Von Neumann and Oskar Morgenstern. Theory of games and economic behavior, 2nd rev. 1947.
- Markus Wulfmeier, Peter Ondruska, and Ingmar Posner. Maximum Entropy Deep Inverse Reinforcement Learning, March 2016. URL <http://arxiv.org/abs/1507.04888>. arXiv:1507.04888 [cs].
- Sibo Yi, Yule Liu, Zhen Sun, Tianshuo Cong, Xinlei He, Jiaxing Song, Ke Xu, and Qi Li. Jailbreak Attacks and Defenses Against Large Language Models: A Survey, August 2024. URL <http://arxiv.org/abs/2407.04295>. arXiv:2407.04295 [cs].
- Chenyang Zhao, Olivier Sigaud, Freek Stulp, and Timothy M. Hospedales. Investigating Generalisation in Continuous Deep Reinforcement Learning, February 2019. URL <http://arxiv.org/abs/1902.07015>. arXiv:1902.07015 [cs].
- Brian D Ziebart, J Andrew Bagnell, and Anind K Dey. Maximum entropy inverse reinforcement learning. 2008.

A Related Work

Transfer learning and AI alignment. Transfer learning is a common paradigm in modern machine learning [Iman et al., 2023]. The most notable example is the use of LLMs pre-trained on large text corpora and then RL fine-tuned to build helpful assistants [Bai et al., 2022a] or powerful reasoners [DeepSeek-AI et al., 2025], but applications of foundation models are also being explored in robotics [Firoozi et al., 2025]. Systems trained via transfer learning are at the frontier of AI developments, with concerns over the safety of powerful AI systems [Amodei et al., 2016, Hubinger et al., 2021, Ngo et al., 2025, Hendrycks et al., 2023] motivating a variety of complex approaches to try and better align these models with human values [Bai et al., 2022b, Guan et al., 2025, Maiya et al., 2025, Rafailov et al., 2023].

However, little is understood about how these complex training pipelines, from pre-training to alignment and reasoning post-training, end up shaping the final values and behaviours of the models they produce. Despite seeming to work at the surface level, research on jailbreaks [Yi et al., 2024], emergent misalignment [Betley et al., 2026, Taylor et al., 2025], and misalignment in more agentic settings [Lynch et al., 2025, Andriushchenko et al., 2025, Naik et al., 2025] show there is still lots more work to be done. That said, alignment approaches that consider the entire training pipeline are starting to be explored [Tice et al., 2026], though we lack a principled theory to understand the empirically observed effects.

One particular failure mode is *goal-misgeneralisation* [Langosco et al., 2023], where even a correctly specified reward function fails to induce the intended behaviour outside of the training distribution. In our paper we try to better understand goal generalisation, and how it is influenced by multi-stage training pipelines.

Inferring the values of agents. In our work, we try to infer the values of AI agents behaviourally, and then try to predict them with just knowledge of the training pipeline. Whilst our methods are simple, extensive work in the subfield of inverse RL (IRL) has detailed and analysed many more approaches for inferring latent values underpinning agentic behaviour, and the reward structures that might incentivise it [Wulfmeier et al., 2016, Skalse and Abate, 2023, Ramachandran and Amir, Hadfield-Menell et al., 2016, Adams et al., 2022, Ziebart et al., 2008, Abbeel and Ng, 2004, Brown et al., 2025]. However, this has mostly been restricted to robotics or game-playing tasks, and focuses on building reward functions, rather than understanding the dynamics of agent value formation.

More recent work on trying to infer the values of frontier AIs includes that of Mazeika et al. [2025], who analyse the preferences of LLMs expressed over various hypothetical scenarios, and try to infer their underlying values. Focussing on more real-world settings, Andriushchenko et al. [2025] analyses how LLMs can engage in harmful behaviours when given access to external tools, and Lynch et al. [2025], Naik et al. [2025] both show agentic models deployed to perform harmless tasks might engage in risky and malicious behaviour to achieve their goals.

Mini et al. [2023] uses mechanistic interpretability [Bereska and Gavves, 2024] to discover motivational-circuits within a maze-solving agent, showing how methods other than evaluating behaviour can be used to infer the value structure of an AI agent. Extending this philosophy, we directly consider the relationship between learnt values and the training pipeline that produced them, and believe this is an untapped well of potential insight.

Generalisation in machine learning. Beyond values and goals, there is a large body of existing literature on how RL systems generalise [Cobbe et al., 2019, Zhao et al., 2019, Kirk et al., 2023], though this is mostly focussed on generalisation of agent capabilities, rather than their values. Prominent works highlight the issue of overfitting, and how to deal with it [Cobbe et al., 2019, 2020]. This motivated our use of procedural generation for our tasks, to force our agents to generalise their capabilities as best they could to novel mazes with unfamiliar goals.

In accordance with our results on value persistence and strengthening, other works have found that early training data can have outsized impacts on the learning of deep RL agents [Nikishin et al., 2022], and that when models are successfully architected to transfer learn successfully, often existing low-level representations are re-used [Rusu et al., 2022]. A related and rapidly growing literature investigates the *loss of plasticity* in continual and multi-stage neural network training—the tendency of networks to lose their ability to adapt to new data as training progresses [Dohare et al., 2024, Lyle

et al., 2024, Nikishin et al., 2023]. Relatedly, Pezeshki et al. [2021] identify *gradient starvation*, whereby easy-to-learn features dominate gradient updates and suppress learning of other informative features—a phenomenon that parallels our finding that salient features inhibit the formation of new values. This has been linked to mechanisms such as the emergence of dormant neurons in deep RL networks [Sokar et al., 2023]. Our findings on value persistence and value inhibition are consistent with these phenomena, and we view our latent policy gradients method as providing a complementary, predictive account of their effects in the specific setting of multi-stage goal-directed RL with compositional goals.

Additionally, our results surrounding the importance of feature salience draws parallels with previous work showing how deep learning models often exploit shortcuts and spurious correlations in their training data in order to achieve high performance on a task, as these are often easier than generalising robustly [Geirhos et al., 2020, Haan et al., 2019]. Contrary to this, more contemporary work suggests that extended training might overcome this effect [Power et al., 2022, Nakkiran et al., 2021], and that if we trained our models for a far greater amount of time then feature saliencies and previously learnt values might become less important in determining generalisation behaviour. For CNNs specifically, prior work has found they have a bias for categorising based on shape rather than colour [Ritter et al., 2017, Feinman and Lake, 2018], consistent with our saliency analysis.

Behavioural psychology and conditioning. Our work has strong connections to several neuroscientific works. Our latent policy gradient method, and the dynamics of value learning it predicts, bears strong resemblance to the Rescorla-Wagner model of Pavlovian conditioning, and similarly predicts dynamics such as blocking and overshadowing [Rescorla, 1972]. Other theories of conditioning, such as Configural theory [Pearce, 1987], diverge from our specific parametrisation (though do not conflict with the intuition behind latent policy gradients at a high level), and might be interesting sources of inspiration for future models of agent values. Finally, several works investigate stimulus generalisation in a way that bears resemblance to how we think about feature confusion, and the non-diagonal entries of the saliency matrix [Shepard, 1987, McLaren and Mackintosh, 2002].

B Experimental Details

B.1 Agent

Our agent is implemented using Stable Baselines 3 [Raffin et al., 2021]. We use the PPO algorithm [Schulman et al., 2017] with a standard CNN-based policy [Mnih et al., 2015], our hyperparameters are given in Table 1. The policy uses a shared CNN feature extractor, and then separate heads for the actor and critic networks respectively:

```
ActorCriticCnnPolicy(
    (features_extractor): NatureCNN(
      (cnn): Sequential(
        (0): Conv2d(3, 32, kernel_size=(8, 8), stride=(4, 4))
        (1): ReLU()
        (2): Conv2d(32, 64, kernel_size=(4, 4), stride=(2, 2))
        (3): ReLU()
        (4): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1))
        (5): ReLU()
        (6): Flatten(start_dim=1, end_dim=-1)
      )
      (linear): Sequential(
        (0): Linear(in_features=9216, out_features=512, bias=True)
        (1): ReLU()
      )
    )
    (action_net): Linear(in_features=512, out_features=4, bias=True)
    (value_net): Linear(in_features=512, out_features=1, bias=True)
)
```

The network has a total of 4.8 million parameters, all of which are randomly initialised for the first stage of training.

Table 1: PPO hyperparameters used for training all agents.

Hyperparameter	Value
Learning rate	0.0001
Steps per rollout	256
Batch size	64
Epochs per update	10
Discount factor (γ)	0.95
GAE lambda	0.9
Clip range	0.2
Entropy coefficient	0.01
Value function coefficient	0.5
Max gradient norm	0.5

B.2 The Maze Environment

To generate mazes, we sample 8×8 grids where each cell is a wall independently with probability 0.2. Mazes with inaccessible vacant squares are discarded and re-generated. The goal object and agent are placed in random vacant cells. Each cell is rendered as a 16×16 pixel image, giving the agent 128×128 RGB observations.

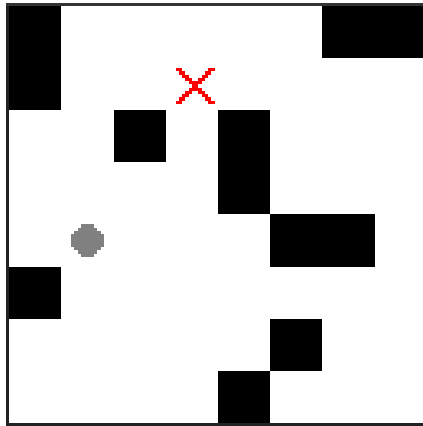


Figure 4: **An example 8x8 maze environment.** The agent is represented by $\bullet$ and the $\times$ is the goal object. Black squares are impassable walls. The agent cannot move off the edges of the maze; the outer wall shown is for illustrative purposes and is not included as part of the agent’s observation. The total observation size is 128×128 pixels with 3 colour channels (RGB).

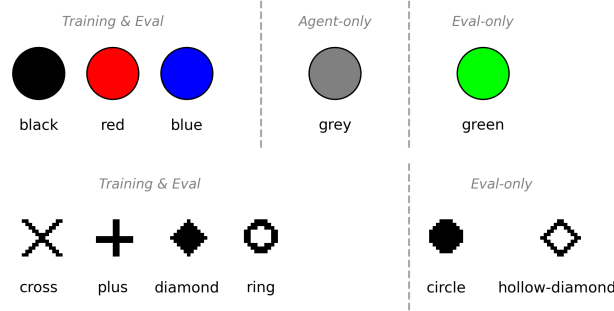


Figure 5: **Visual features used in the Maze environment.** **Colours** (top): Three colours (black, red, blue) are used for goal objects during both training and evaluation; grey is used exclusively to render the agent; green appears only in evaluation environments to test generalisation to novel colours. All red, blue, and green use a single RGB input channel. **Shapes** (bottom): Four shapes (cross, plus, diamond, ring) are used during training and evaluation; circle and hollow-diamond appear only in evaluation to test generalisation to novel shapes. Each cell of the maze is rendered as a 16×16 pixel image, thus the rendered shapes have a low resolution.

C Methodological Details

C.1 Elo Score Computation

Under the Elo model, each object receives a scalar score V , and the probability that an agent selects object A over object B is given by $1/(1 + 10^{-(V_A - V_B)/400})$. This is equivalent to a logistic function of the score difference with a growth rate of $\frac{\ln 10}{400}$, and can be derived from the Bradley-Terry model of pairwise comparisons [Bradley and Terry, 1952].

Since our evaluation data includes a third outcome of the episode terminating without either object being reached, we convert each three-way observation into three pairwise comparisons. This is done by masking out each outcome in turn and normalising the remaining two rates. A “no goal” baseline is included as a competitor in the Elo system. After fitting, we shift all scores so that the no-goal score is zero, utilising Elo’s invariance to uniform shifts. This ensures that each object’s Elo score reflects both its relative preference and its absolute likelihood of being pursued, while remaining comparable across agents.

C.2 Latent Policy Gradient Fitting

We detail the model fitting procedure in Algorithm 1. The outer loop optimises the hyperparameters S , τ , and w_0 ¹⁰ by minimising the modelling loss (Equation (1)) via gradient descent. To construct $\mathcal{D}_{\text{train}}$ we add a separate entry for each evaluation environment of each agent, so for our 298 agents each with 276 evaluation environments we obtain 82k datapoints.

For each training example, Algorithm 2 computes the latent weights w_Ω by simulating gradient ascent with $n_{\text{integration-steps}} = 100$ on the modified policy objective (Equation (2)) through each stage of the training pipeline. Since this inner simulation is differentiable with respect to the hyperparameters, we’re able to optimise them with gradient descent in the outer loop.

The algorithm is generic across different validation scenarios. For full-data fitting, $\mathcal{D}_{\text{train}} = \mathcal{D}_{\text{eval}}$. For K-fold cross-validation, agents are partitioned and the procedure is repeated K times with different train/eval splits. For transfer evaluations (single-stage $\rightarrow$ multi-stage, or no-distractor $\rightarrow$ distractor), training uses only the simpler pipelines while evaluation uses the more complex ones.

We have given Algorithm 1 in the form of full-batch gradient descent for simplicity, however, in practice we mini-batch with batch size 64 and use the Adam optimiser [Kingma, 2014] with a learning rate of 0.03 and a β_1 and β_2 of 0.9 and 0.999 respectively. We perform only a single epoch as we found it gave similar performance to more epochs, but was much faster.

¹⁰In practice, we store $\log \tau$ rather than τ directly, ensuring $\tau > 0$ and improving numerical stability during optimisation.

Algorithm 1 Fitting the Latent Policy Gradient Model

Require: Training data $\mathcal{D}_{\text{train}} = \{(\Omega_i, \phi_i^{(a)}, \phi_i^{(b)}, \hat{\Pi}_i)\}_{i=1}^N$, where Ω_i is a training pipeline, $(\phi_i^{(a)}, \phi_i^{(b)})$ is an evaluation goal pair, and $\hat{\Pi}_i$ is the observed choice distribution

Require: Evaluation data $\mathcal{D}_{\text{eval}}$ (same format)

Require: Learning rate η , number of epochs E , number of features n

Ensure: Fitted saliency matrix S , temperature τ , initial value w_0 ; evaluation loss $\mathcal{L}_{\text{eval}}$

- 1: **Initialise:** $S \leftarrow I_n$ (upper-triangular), $\tau \leftarrow 1$, $w_0 \leftarrow 0$
- 2: **for** epoch = 1, ..., E **do**
- 3: $\mathcal{L}_{\text{train}} \leftarrow 0$
- 4: **for** each $(\Omega, \phi^{(a)}, \phi^{(b)}, \hat{\Pi}) \in \mathcal{D}_{\text{train}}$ **do**
- 5: $w_\Omega \leftarrow \text{SIMULATEPIPELINE}(\Omega, S, \tau, w_0)$ {Alg. 2}
- 6: $\Pi_\Omega \leftarrow \text{softmax}([\phi^{(a)} \cdot S w_\Omega, \phi^{(b)} \cdot S w_\Omega, 0])$
- 7: $\mathcal{L}_{\text{train}} \leftarrow \mathcal{L}_{\text{train}} + D_{\text{KL}}(\hat{\Pi} \parallel \Pi_\Omega)$
- 8: **end for**
- 9: Update S, τ, w_0 via gradient descent on $\mathcal{L}_{\text{train}}$ with learning rate η
- 10: **end for**
- 11: **Evaluate:** $\mathcal{L}_{\text{eval}} \leftarrow 0$
- 12: **for** each $(\Omega, \phi^{(a)}, \phi^{(b)}, \hat{\Pi}) \in \mathcal{D}_{\text{eval}}$ **do**
- 13: $w_\Omega \leftarrow \text{SIMULATEPIPELINE}(\Omega, S, \tau, w_0)$
- 14: $\Pi_\Omega \leftarrow \text{softmax}([\phi^{(a)} \cdot S w_\Omega, \phi^{(b)} \cdot S w_\Omega, 0])$
- 15: $\mathcal{L}_{\text{eval}} \leftarrow \mathcal{L}_{\text{eval}} + D_{\text{KL}}(\hat{\Pi} \parallel \Pi_\Omega)$
- 16: **end for**
- 17: $\mathcal{L}_{\text{eval}} \leftarrow \mathcal{L}_{\text{eval}} / |\mathcal{D}_{\text{eval}}|$
- 18: **return** $S, \tau, w_0, \mathcal{L}_{\text{eval}}$

Algorithm 2 SIMULATEPIPELINE: Compute Latent Weights via Policy Gradient Ascent

Require: Pipeline $\Omega = \{(\phi^{(g_t)}, \phi^{(d_t)})\}_{t=1}^T$ with T stages, each having goal features $\phi^{(g_t)}$ and optional distractor features $\phi^{(d_t)}$

Require: Saliency matrix S , temperature τ , initial value w_0

Ensure: Latent weights w

- 1: $w \leftarrow w_0 \mathbf{1}$
- 2: **for** stage $t = 1, \dots, T$ **do**
- 3: **for** step = 1, ..., $n_{\text{integration-steps}}$ **do**
- 4: $v^g \leftarrow \phi^{(g_t)} \cdot S w$ {Goal value}
- 5: **if** distractor $\phi^{(d_t)}$ present **then**
- 6: $v^d \leftarrow \phi^{(d_t)} \cdot S w$ {Distractor value}
- 7: $\pi_G \leftarrow \exp(v^g) / (1 + \exp(v^g) + \exp(v^d))$
- 8: **else**
- 9: $\pi_G \leftarrow \sigma(v^g)$ {Goal probability}
- 10: **end if**
- 11: $J \leftarrow \pi_G + \tau h(\pi_G)$ {Modified policy objective, Equation (2)}
- 12: $w \leftarrow w + \nabla_w J$ {Gradient ascent step}
- 13: **end for**
- 14: **end for**
- 15: **return** w

D Fitted Model Parameters

Here we present the fitted hyperparameters for our proposed model when fit on all our data.

For the entropy regularisation coefficient, τ , we obtain a value of 0.698, showing our agents were generally modelled as moderately effective at achieving their goals. For the initial latent value, w_0 , we obtain a value of -0.372 , indicating that agents begin with a slight prior against pursuing any goal, consistent with untrained agents not yet having learned goal-directed behaviour. The entries of S and SS^T are given in Tables 2 and 3 respectively. S alone is not very interpretable, but as discussed in Section 5.5, SS^T induces an interpretable natural similarity metric over features. In agreement with the empirical results, we see that shapes are stronger than colours at driving generalisation, with cross being by far the strongest. Hollow-diamond is the weakest shape and black is the weakest feature overall.

Table 2: **Saliency Matrix**, S (upper-triangular). This maps between latent space and goal-feature space.

S_{ij}	$\leftarrow w \rightarrow$									
black	0.994	0.349	0.289	0.142	-0.253	0.065	0.461	0.092	0.239	0.157
blue	0	2.088	-0.379	-0.318	0.188	0.099	0.114	0.341	0.206	0.133
green	0	0	1.549	-0.394	0.131	0.325	0.097	0.116	0.211	0.076
red	0	0	0	2.080	0.044	0.011	0.183	0.216	0.277	0.181
circle	0	0	0	0	1.445	-0.084	0.689	0.417	0.115	0.100
cross	0	0	0	0	0	2.794	-0.476	-0.468	-0.536	-0.587
diamond	0	0	0	0	0	0	1.781	0.157	0.280	-0.476
hollow-dia.	0	0	0	0	0	0	0	1.157	-0.158	0.294
plus	0	0	0	0	0	0	0	0	2.428	-0.470
ring	0	0	0	0	0	0	0	0	0	2.478

Table 3: **Induced similarity metric**, SS^T . This captures the effective learning rate interactions between features. Diagonal entries (shown in bold) represent the relative strengths of features, while off-diagonal entries represent cross-feature interactions.

$[SS^T]_{ij}$	black	blue	green	red	circle	cross	diamond	hollow-dia.	plus	ring
black	1.585	0.685	0.498	0.485	0.028	-0.300	0.828	0.114	0.507	0.389
blue	0.685	4.838	-0.300	-0.477	0.522	-0.125	0.250	0.402	0.437	0.331
green	0.498	-0.300	2.750	-0.695	0.309	0.649	0.213	0.123	0.476	0.189
red	0.485	-0.477	-0.695	4.516	0.329	-0.414	0.352	0.259	0.587	0.449
circle	0.028	0.522	0.309	0.329	2.767	-0.877	1.277	0.493	0.231	0.249
cross	-0.300	-0.125	0.649	-0.414	-0.877	8.882	-0.792	-0.630	-1.025	-1.456
diamond	0.828	0.250	0.213	0.352	1.277	-0.792	3.500	-0.003	0.903	-1.178
hollow-dia.	0.114	0.402	0.123	0.259	0.493	-0.630	-0.003	1.449	-0.522	0.729
plus	0.507	0.437	0.476	0.587	0.231	-1.025	0.903	-0.522	6.115	-1.166
ring	0.389	0.331	0.189	0.449	0.249	-1.456	-1.178	0.729	-1.166	6.141

E Latent Dimension Analysis

We sweep the latent dimension d (the size of the weight vector w) from 1 to 32, keeping all other hyperparameters at their default values. Figure 6 shows that loss decreases monotonically up to $d = 8$ and plateaus thereafter, with no benefit from dimensions beyond $n_{\text{features}} = 10$. We use $d = 10$ in the main text, which sits at the plateau and matches the number of object features, giving each latent dimension a natural interpretation. The clean plateau suggests that the underlying preference structure has moderate effective dimensionality and that the choice of d is not sensitive provided $d \geq n_{\text{features}}$.

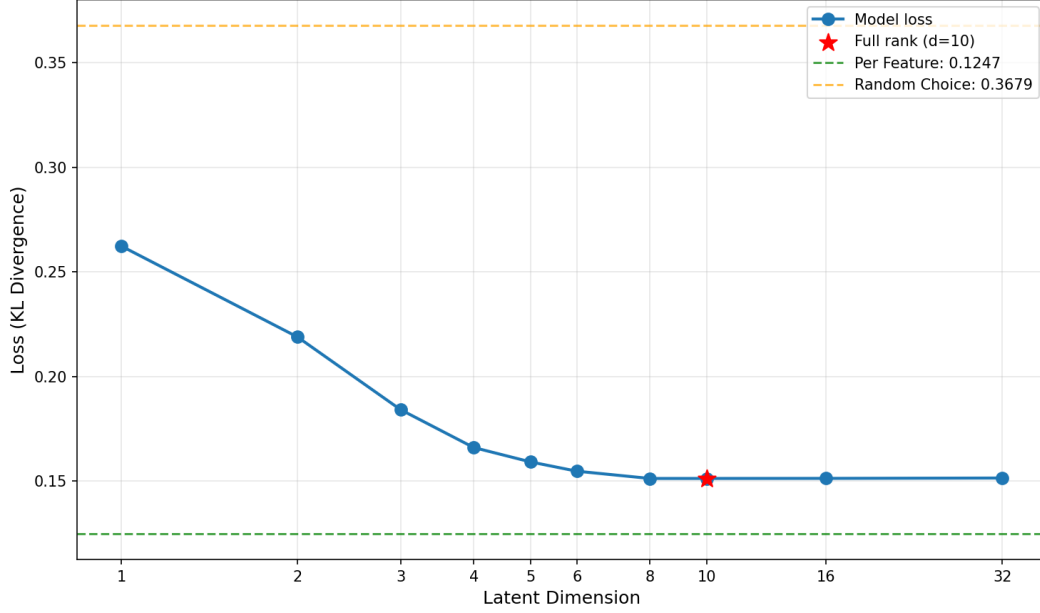


Figure 6: Model loss (KL divergence) as a function of the latent dimension d . The dashed lines show the uniform random baseline (upper) and the per-agent per-feature lower bound (lower). Loss plateaus at $d = 8$; the red star marks $d = 10 = n_{\text{features}}$, the value used in the main text.

F Additional Model Evaluation Metrics

Throughout this appendix, we evaluate model predictions using the following metrics. We report these in two variants: over the full **three-way** distribution (goal a , goal b , neither), which is the distribution our models are trained to predict; and over a renormalised **two-way** distribution (goal a vs goal b), which removes the null-goal component and isolates how well the model captures *relative* goal preferences, independent of its ability to predict the overall goal-reaching rate.

For a single evaluation example with observed distribution $\hat{p} = (\hat{p}_1, \dots, \hat{p}_K)$ and predicted distribution $q = (q_1, \dots, q_K)$ where K is either 3 (three-way) or 2 (two-way):

KL divergence (training objective in the main text):

$$D_{\text{KL}}(\hat{p}||q) = \sum_{k=1}^K \hat{p}_k \log \frac{\hat{p}_k}{q_k}. \quad (7)$$

Total variation (TV) distance, which provides a scale-interpretable bound on the maximum prediction error for any event:

$$\text{TV}(\hat{p}, q) = \frac{1}{2} \sum_{k=1}^K |\hat{p}_k - q_k|. \quad (8)$$

Brier score, the mean squared error between predicted and observed probabilities, which penalises confident incorrect predictions:

$$\text{BS}(\hat{p}, q) = \frac{1}{K} \sum_{k=1}^K (\hat{p}_k - q_k)^2. \quad (9)$$

Directional accuracy, the fraction of non-trivial evaluation pairs¹¹ for which the model correctly predicts which goal the agent prefers:

$$\text{Dir. Acc.} = \frac{1}{|\mathcal{D}'|} \sum_{i \in \mathcal{D}'} \mathbf{1} \left[\text{sign}(q_1^{(i)} - q_2^{(i)}) = \text{sign}(\hat{p}_1^{(i)} - \hat{p}_2^{(i)}) \right], \quad (10)$$

where $\mathcal{D}'$ is the set of examples exceeding the directional threshold. All metrics are averaged over all evaluation examples. Lower is better for KL, TV, and Brier; higher is better for directional accuracy.

F.1 Elo Holdout Validation

To validate that Elo scores reliably summarise the pairwise preference data, we perform 4-fold cross-validation. For each agent, we partition the 276 pairwise comparisons into 4 folds, fit Elo scores on 3 folds, and predict win probabilities on the held-out fold using $P(\text{goal}_a) = 1/(1 + 10^{-(V_a - V_b)/400})$. We report the mean and standard error of each metric across all 298 agents.

Table 4: Elo 4-fold cross-validation results (297 agents). Elo scores fitted on 75% of pairwise comparisons accurately predict the held-out 25%.

Metric	Mean	SE
KL divergence	0.0511	0.0010
Total variation distance	0.0922	0.0007
Brier score	0.0339	0.0006
Directional accuracy	89.73%	0.18%

As shown in Table 4, Elo scores are a strong predictor of held-out pairwise outcomes: the model correctly predicts which goal is preferred in nearly 90% of non-trivial matchups, with low KL divergence and well-calibrated probability estimates (Brier score 0.034).

F.2 Elo Scores vs Model-Predicted Values

As a further validation that our model captures meaningful preference structure, we compare each agent’s empirical Elo scores with the values predicted by our proposed model. Figure 7 shows the unnormalised comparison, where each point is an (agent, goal) pair. The strong positive correlation (Spearman $\rho = 0.789$, $R^2 = 0.676$, $n = 7,152$) confirms that the model’s predicted values track the empirically observed preference rankings. This plot reflects the full three-way evaluation: the absolute value levels encode information about the agent’s overall goal-reaching rate (analogous to the no-goal probability), in addition to relative goal preferences.

Figure 8 shows the same data after subtracting the per-agent mean from both Elo scores and model values. This zero-mean normalisation removes the per-agent baseline, isolating how well the model captures *relative* goal preferences within each agent—analogous to the two-way evaluation that ignores no-goal probabilities. The improved correlation (Spearman $\rho = 0.805$, $R^2 = 0.699$) indicates that the model captures relative preferences somewhat better than absolute value levels. In both plots, points are coloured by goal colour and shaped by goal shape, and the visual clustering confirms that the model learns meaningful colour and shape representations.

F.3 Supplementary Evaluation Metrics

Table 5 gives the exact numerical values corresponding to Figure 2 in the main text.

In the main text, we report the KL divergence as our primary evaluation metric, since this is the objective used to fit the model’s hyperparameters. To provide additional confidence that the model’s performance is not an artefact of optimising for a single metric, we report all four metrics defined above in Tables 6 and 7. These metrics are computed post-hoc on the full dataset using saved model parameters, and are not used during hyperparameter fitting.

¹¹We exclude pairs where the observed normalised rate difference is less than 10 percentage points, as these are near-ties where directional prediction is not meaningful.

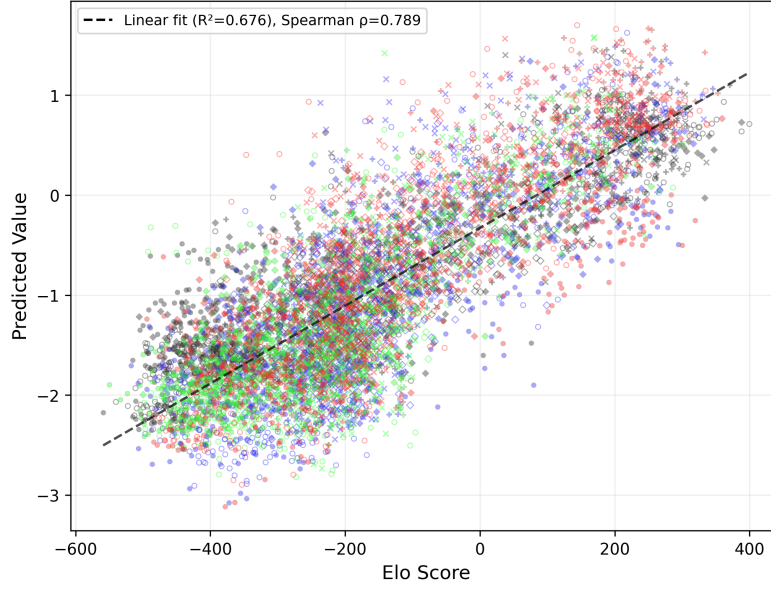


Figure 7: Empirical Elo scores vs model-predicted values for all 298 agents across 24 goals. Each point is an (agent, goal) pair, coloured by the goal’s colour and shaped by the goal’s shape.

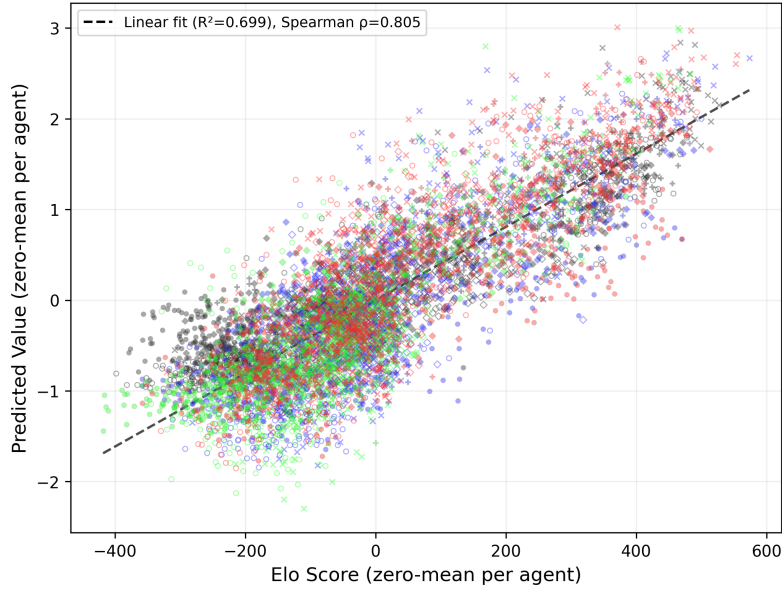


Figure 8: Zero-mean normalised Elo scores vs model-predicted values. Per-agent means are subtracted from both axes, isolating relative goal preferences within each agent.

Two-way vs three-way evaluation. Table 6 reports metrics over the full three-way distribution (goal a , goal b , neither), which is the distribution our model is trained to predict. Table 7 reports metrics after renormalising to a two-way distribution (goal a vs goal b), removing the null-goal component. This isolates how well the model captures *relative* goal preferences, independent of its ability to predict the overall goal-reaching rate.

The model ranking is consistent across all metrics and both evaluation variants, with the proposed model achieving the best performance on every metric. The directional accuracy of 87.71% (three-way) indicates that the model correctly predicts the preferred goal in nearly 9 out of 10 non-trivial matchups.

Table 5: Average modelling loss (Equation (1)) across our proposed model, four alternatives, a baseline, and two lower bounds, for a variety of evaluations. Lower is better, **best** model in column is shown in bold. For K-fold Cross Validation we give the standard error computed over the 4 folds. For the latter two evaluations, their hyperparameters are fit on training pipelines matching the upper descriptor, but then we report the modelling loss evaluated over training pipelines matching the lower descriptor. The per-agent lower bounds fit values directly to each agent’s observed preferences rather than predicting them from training pipelines, so they are only applicable to the Full Fit evaluation.

Method	Hyper-parameters	Full Fit	K-Fold Cross-Validation	Single-Stage Fit Two-Stage Loss	No Distractors Fit Distractors Loss
Proposed Model	57	0.1513	0.1532 $\pm$ 0.0031	0.1851	0.1535
Simultaneous	57	0.1534	0.1554 $\pm$ 0.0028	0.2018	0.1577
Quadratic	67	0.1605	0.1637 $\pm$ 0.0036	0.2009	0.1653
Diagonal S	12	0.1765	0.1786 $\pm$ 0.0033	0.2069	0.1762
Memoryless	57	0.2041	0.2073 $\pm$ 0.0059	0.2644	0.1900
Uniform Random (Baseline)	0	0.3679	0.3679 $\pm$ 0.0000	0.3776	0.3656
Per-Agent Per-Goal (Lower Bound)	7,152	0.0921	N/A	N/A	N/A
Per-Agent Per-Feature (Lower Bound)	2,980	0.1247	N/A	N/A	N/A

Table 6: Post-hoc evaluation metrics over the **three-way** distribution (Full Fit). KL divergence is the training objective; other metrics are reported for validation.

Method	KL	TV Distance	Brier Score	Dir. Accuracy
Proposed Model	0.1536	0.2036	0.0993	87.71%
Simultaneous	0.1558	0.2059	0.1006	87.10%
Quadratic	0.1627	0.2105	0.1054	85.61%
Diagonal S	0.1789	0.2242	0.1164	84.16%
Memoryless	0.2070	0.2373	0.1328	77.28%
Uniform Random	0.3679	0.3745	0.2459	0.00%

Table 7: Post-hoc evaluation metrics over the **two-way** (renormalised) distribution (Full Fit). This isolates relative goal preference prediction, removing the null-goal component.

Method	KL	TV Distance	Brier Score	Dir. Accuracy
Proposed Model	0.0750	0.1323	0.0597	79.03%
Simultaneous	0.0779	0.1355	0.0622	78.58%
Diagonal S	0.0823	0.1415	0.0684	75.26%
Quadratic	0.0856	0.1412	0.0676	77.29%
Memoryless	0.1057	0.1612	0.0895	71.69%
Uniform Random	0.1423	0.2107	0.1270	0.00%

G Proofs for Latent Policy Gradient Analysis

G.1 Derivation of the Latent Gradient (Equation 4)

We derive the gradient of the modified policy objective with respect to the latent variables $\mathbf{w}$.

Case 1: No distractor. Recall the modified policy objective:

$$J = \Pi \left(\phi^{(g)} \middle| \phi^{(g)}, \mathbf{0} \right) + \tau h \left(\Pi \left(\phi^{(g)} \middle| \phi^{(g)}, \mathbf{0} \right) \right) \quad (11)$$

where $h(p) = -p \log p - (1-p) \log(1-p)$ is the binary entropy function. Under our linear Boltzmann-rational parametrisation, $\Pi = \sigma(v^g)$ where $v^g = \phi^{(g)} \cdot S\mathbf{w}$.

We compute the gradient via the chain rule:

$$\nabla_{\mathbf{w}} J = \frac{\partial J}{\partial \Pi} \cdot \frac{\partial \Pi}{\partial v^g} \cdot \frac{\partial v^g}{\partial \mathbf{w}} \quad (12)$$

Step 1: The derivative of the entropy function is:

$$\frac{dh}{dp} = -\log p - 1 + \log(1-p) + 1 = \log \left(\frac{1-p}{p} \right) \quad (13)$$

Therefore:

$$\frac{\partial J}{\partial \Pi} = 1 + \tau \log \left(\frac{1-\Pi}{\Pi} \right) \quad (14)$$

Since $\Pi = \sigma(v^g) = \frac{e^{v^g}}{1+e^{v^g}}$, we have $\frac{1-\Pi}{\Pi} = e^{-v^g}$, and thus:

$$\frac{\partial J}{\partial \Pi} = 1 - \tau v^g \quad (15)$$

Step 2: The derivative of the sigmoid function is:

$$\frac{\partial \Pi}{\partial v^g} = \sigma(v^g)(1 - \sigma(v^g)) \quad (16)$$

Step 3: From $v^g = \phi^{(g)} \cdot S\mathbf{w}$:

$$\frac{\partial v^g}{\partial \mathbf{w}} = S^T \phi^{(g)} \quad (17)$$

Combining these three steps:

$$\nabla_{\mathbf{w}} J = (1 - \tau v^g) \sigma(v^g) (1 - \sigma(v^g)) S^T \phi^{(g)} \quad (18)$$

as claimed. $\square$

Case 2: With distractor. When a distractor object with features $\phi^{(d)}$ is present, the agent chooses between three options: the goal, the distractor, and doing nothing. The natural extension of our Boltzmann-rational parametrisation is:

$$\Pi \left(\phi^{(g)} \middle| \phi^{(g)}, \phi^{(d)}, \mathbf{0} \right) = \frac{\exp(v^g)}{\exp(v^g) + \exp(v^d) + 1} \quad (19)$$

where $v^g = \phi^{(g)} \cdot S\mathbf{w}$ and $v^d = \phi^{(d)} \cdot S\mathbf{w}$ are the values assigned to the goal and distractor respectively.

For notational convenience, let $\Pi^g = \Pi \left(\phi^{(g)} \middle| \phi^{(g)}, \phi^{(d)}, \mathbf{0} \right)$ denote the probability of choosing the goal. The modified policy objective becomes:

$$J = \Pi^g + \tau h(\Pi^g) \quad (20)$$

As before, we have:

$$\frac{\partial J}{\partial \Pi^g} = 1 + \tau \log \left(\frac{1 - \Pi^g}{\Pi^g} \right) \quad (21)$$

To compute $\frac{\partial \Pi^g}{\partial \mathbf{w}}$, let $Z = \exp(v^g) + \exp(v^d) + 1$ be the partition function. Then:

$$\frac{\partial \Pi^g}{\partial v^g} = \frac{\exp(v^g) \cdot Z - \exp(v^g) \cdot \exp(v^g)}{Z^2} = \Pi^g(1 - \Pi^g) \quad (22)$$

$$\frac{\partial \Pi^g}{\partial v^d} = \frac{0 - \exp(v^g) \cdot \exp(v^d)}{Z^2} = -\Pi^g \Pi^d \quad (23)$$

where $\Pi^d = \frac{\exp(v^d)}{Z}$ is the probability of choosing the distractor.

Therefore:

$$\frac{\partial \Pi^g}{\partial \mathbf{w}} = \frac{\partial \Pi^g}{\partial v^g} \frac{\partial v^g}{\partial \mathbf{w}} + \frac{\partial \Pi^g}{\partial v^d} \frac{\partial v^d}{\partial \mathbf{w}} \quad (24)$$

$$= \Pi^g(1 - \Pi^g) S^T \phi^{(g)} - \Pi^g \Pi^d S^T \phi^{(d)} \quad (25)$$

$$= \Pi^g \left[(1 - \Pi^g) S^T \phi^{(g)} - \Pi^d S^T \phi^{(d)} \right] \quad (26)$$

Combining with $\frac{\partial J}{\partial \Pi^g}$, the full gradient is:

$$\nabla_{\mathbf{w}} J = \left(1 + \tau \log \left(\frac{1 - \Pi^g}{\Pi^g} \right) \right) \Pi^g \left[(1 - \Pi^g) S^T \phi^{(g)} - \Pi^d S^T \phi^{(d)} \right] \quad (27)$$

Note that in the distractor case, the gradient is no longer purely in the direction $S^T \phi^{(g)}$, but contains an additional component in the direction $S^T \phi^{(d)}$. This reflects the fact that training with a distractor present affects how the agent values features associated with the distractor. $\square$

G.2 Analytic Solution for Equilibrium Weights (Equation 5)

We derive the closed-form expression for the weights after convergence to equilibrium in the no-distractor case.

From the gradient expression derived above, the updates to $\mathbf{w}$ are always in the direction $S^T \phi^{(g)}$. Therefore, starting from initial weights $\mathbf{w}_0$, the final weights must take the form:

$$\mathbf{w} = \mathbf{w}_0 + \alpha S^T \phi^{(g)} \quad (28)$$

for some scalar $\alpha \in \mathbb{R}$.

At equilibrium, setting $\nabla_{\mathbf{w}} J = 0$ and noting that $\sigma(v^g)(1 - \sigma(v^g)) > 0$ for finite v^g , we require:

$$1 - \tau v^g = 0 \implies v^g = \tau^{-1} \quad (29)$$

Substituting the equilibrium condition $v^g = \phi^{(g)} \cdot S\mathbf{w} = \tau^{-1}$:

$$\phi^{(g)} \cdot S \left(\mathbf{w}_0 + \alpha S^T \phi^{(g)} \right) = \tau^{-1} \quad (30)$$

Expanding:

$$\phi^{(g)} \cdot S\mathbf{w}_0 + \alpha \phi^{(g)} \cdot S S^T \phi^{(g)} = \tau^{-1} \quad (31)$$

Noting that $\phi^{(g)} \cdot S S^T \phi^{(g)} = (S^T \phi^{(g)})^T (S^T \phi^{(g)}) = \|S^T \phi^{(g)}\|_2^2$, we solve for α :

$$\alpha = \frac{\tau^{-1} - \phi^{(g)} \cdot S\mathbf{w}_0}{\|S^T \phi^{(g)}\|_2^2} \quad (32)$$

Substituting back:

$$\mathbf{w} = \mathbf{w}_0 + \left(\frac{\tau^{-1} - \phi^{(g)} \cdot S\mathbf{w}_0}{\|S^T \phi^{(g)}\|_2^2} \right) S^T \phi^{(g)} \quad (33)$$

as claimed. $\square$

H Supplementary Figures

H.1 Detailed Empirical Analysis

This section provides the full figures and detailed analysis supporting the empirical findings summarised in Section 4.2.

Feature saliency. Appendix H.1 illustrates how different training goals lead to different generalisation patterns. The agent trained on $\times$ strongly prefers $\times$ -shaped objects over red-coloured objects, whereas the agent trained on $\blacklozenge$ strongly prefers red-coloured objects and only weakly prefers $\blacklozenge$ -shaped objects. Appendix H.1 quantifies this effect across all single-stage agents: for each feature, we average the marginalised Elo over all agents trained on goals containing that feature. Shape tends to drive generalisation more strongly than colour, with $\times$ -shape being the most salient and $\blacklozenge$ -shape the least. Black—the colour of the maze walls—has the lowest average marginalised Elo, as agents trained on black goals generalise over shape rather than colour.

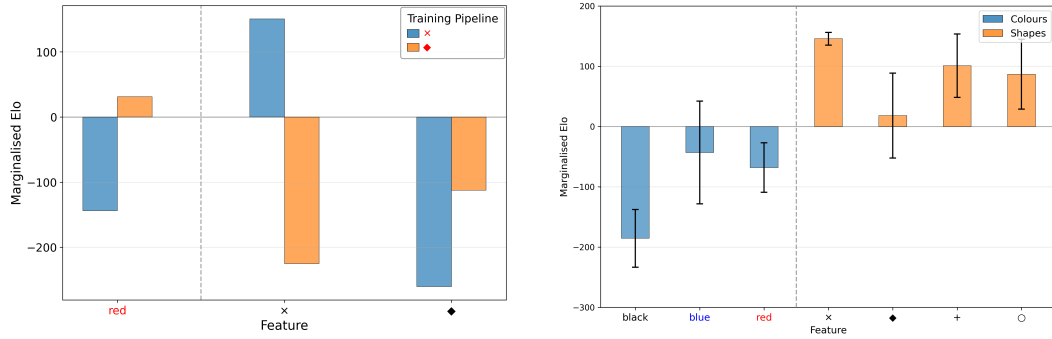


Figure 9: **Left: Generalisation differences between $\times$ and $\blacklozenge$ training goals.** Marginalised feature Elo scores for the red, $\times$ -shape, and $\blacklozenge$ -shape features for the single-stage agent trained on $\times$, and the single-stage agent trained on $\blacklozenge$. **Right: Some features drive generalisation more strongly and are more salient to the model.** Average marginalised Elo with standard error for each feature across agents that have been trained on a goal containing that feature. The average is taken over all single-stage runs without distractors.

Feature cross-saliency and confusion. We also computed the marginalised Elo for each pair of features, measuring the marginalised Elo of single-stage agents for one feature if they were trained on a goal containing the other. Figure 10 shows this cross-saliency matrix. We observe *feature confusion*, where training on one feature can lead to valuing or de-valuing another—a phenomenon captured by the off-diagonal entries of the saliency matrix S in our latent policy gradients model (Section 5).

Value persistence. Appendix H.1 shows a case study: an agent first trained on $\blacklozenge$ and then $\times$ ($\blacklozenge \rightarrow \times$) retains strong preferences for blue-coloured and $\blacklozenge$ -shaped objects, in addition to valuing $\times$ -shaped objects, compared to an agent trained only on $\times$. Appendix H.1 confirms this pattern across all two-stage pipelines: features present only in the first stage’s goal have much higher downstream Elo than controls where that feature is substituted. This holds both when the two goals share a feature and when they do not.

Feature diversity and goal pursuit. Agents trained on more unique features generalise to pursue more objects. Whilst agents typically don’t pursue most objects (corresponding to negative Elos on average), this effect reduces with longer and more feature-diverse training pipelines (Figure 12).

Value strengthening and inhibition. Appendix H.1 shows that subsequently training a $\blacklozenge$ agent on $\times$ does **not** cause it to value $\times$ -shape nearly as much as a single-stage $\times$ agent does. Instead, its value for red is strengthened, and the $\times$ -shape value is much weaker than it would have been without prior training. Appendix H.1 confirms this across all two-stage pipelines: shared features are strengthened (left pair), while the non-shared feature in the second goal is inhibited (right pair).

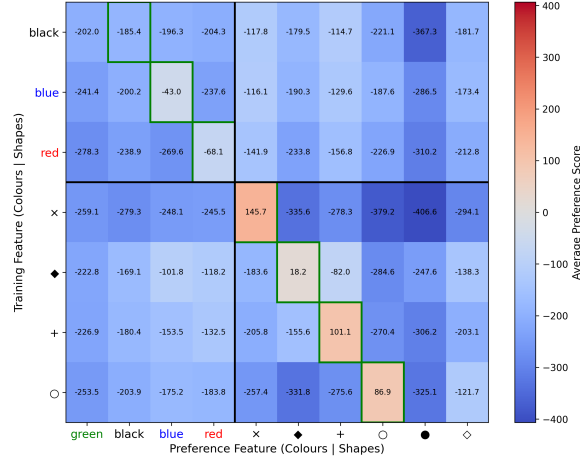


Figure 10: **Training on one feature can lead to another being valued more or less strongly than average.** Rows indicate a feature being trained on and columns indicate a feature being evaluated, the value being the average preference score for goals containing the evaluation feature across models trained on goals containing the training feature. The average is taken over all single-stage runs without distractors.

Ordering effects. A corollary of value strengthening and inhibition is that when goals share a feature, flipping the order of training stages can significantly change the agent’s final values. Figure 14 shows this effect: when there is a shared feature, the **x**-shape feature is often much less likely to be valued when it is present in the second goal vs. the first. We do not observe strong ordering effects when there is no shared feature between the two goals.

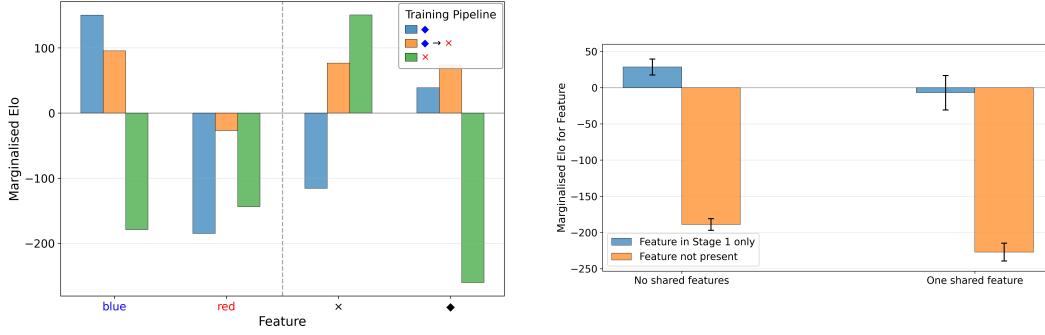


Figure 11: **Left: ♦-shape and blue values persist after training on X.** Marginalised feature Elo scores for red, blue, ♦-shape, and X-shape features for three different agents: single-stage agent trained on ♦; a two-stage agent, ♦→X; a single-stage agent trained just on X. **Right: Values for early training objectives persist.** Marginalised Elo with standard error for features that are only in the first goal in two-stage training pipelines compared to pipelines where they are not present at any stage. We measure this by averaging the marginalised Elo for a given feature across agents with that feature only in their first goal, and comparing this to agents with the same training pipeline aside from the feature of interest, which is substituted for something else. *E.g.*, the blue value of X→♦ is compared with that of X→♦ and X→♦ to determine how much a blue-coloured first goal causes agents to value blue-coloured objects. Here we only consider two-stage pipelines without distractors. On the left of each pair we show this restricted to pipelines whose goals don't share any features, and on the right we restrict it to pipelines where the stage's goals share some other feature than the one being measured. Value persistence is shown by the large differences between the bars within each pair—when a feature is only present in the first stage, the downstream value for that feature is much higher than would be expected if that feature was not present.

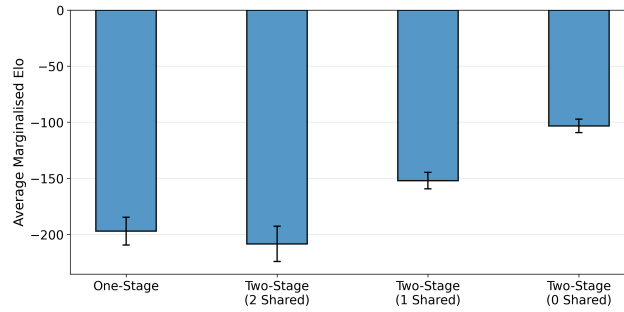


Figure 12: **Agents trained on more diverse feature sets pursue more goals.** Marginalised Elo with standard error across all features for agents that have been trained on a different number of total unique features. Single-stage pipelines always have two unique features in their goals. Two-stage pipelines can have goals which share both, one, or neither of their features.

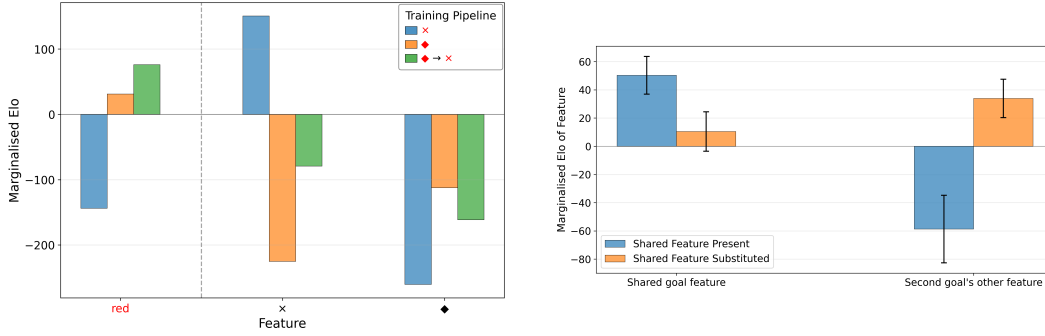


Figure 13: **Left: Training on $\blacklozenge \rightarrow \times$ does not cause a strong value for $\times$ -shape.** Marginalised feature Elo scores for **red**, $\times$ -shape, and $\blacklozenge$ -shape for three different agents: a single-stage agent trained just on a $\times$; a single-stage agent trained on a $\blacklozenge$; a two-stage agent trained on $\blacklozenge \rightarrow \times$. **Right: Repeated goal features' values are strengthened, and inhibit new values forming.** The left bars within each pair show the marginalised Elo with standard error for the two features of the last goal in two-stage training pipelines where a feature is shared between the two goals. The left group of bars plots the marginalised Elo of the shared feature itself, whereas the right group of bars plots the marginalised Elo of the other feature in the second stage goal. The right bars within each pair show the marginalised Elo for the same features but averaged over training pipelines where they are no longer present in the first stage goal (controlling for the other features in the training pipeline). Here we only consider two-stage pipelines without distractors. In the left pair we see that when a feature is shared between two training stages, its value is strengthened, and much higher than if it were only present in the second stage. In the right pair we see that when a feature is shared between two training stages, the other non-shared feature in the second stage goal is valued much less than if the first stage goal did not share any features with the second stage goal.

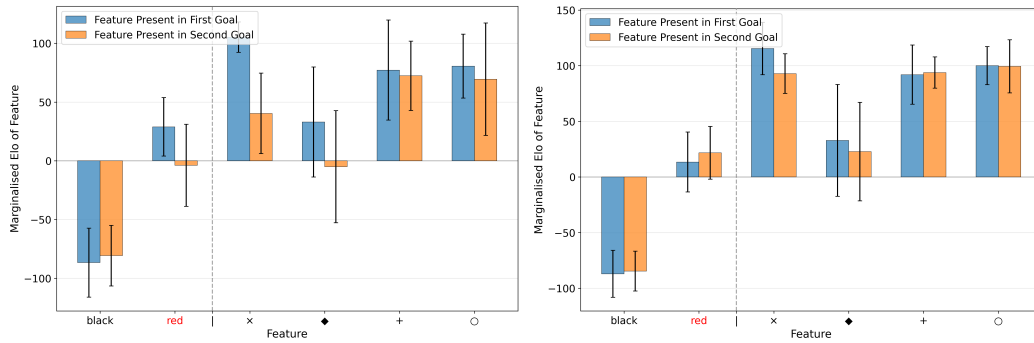


Figure 14: **Generalisation behaviour is sensitive to the order of the training objectives when they share a feature.** Marginalised Elo with standard error for each feature stratified by when that feature is present in the first training goal compared to when it is present in the second training goal. Elo is marginalised over pairs of two-stage pipelines without distractors where they are each others reverse. **Left:** Cases where there is a shared feature between the two goals (e.g., the pair $\blacklozenge \rightarrow \blacklozenge$ and $\blacklozenge \rightarrow \blacklozenge$). **Right:** Cases where there are no shared features between the two goals (e.g., the pair $\blacklozenge \rightarrow \times$ and $\times \rightarrow \blacklozenge$).

H.2 Agent Training Curves

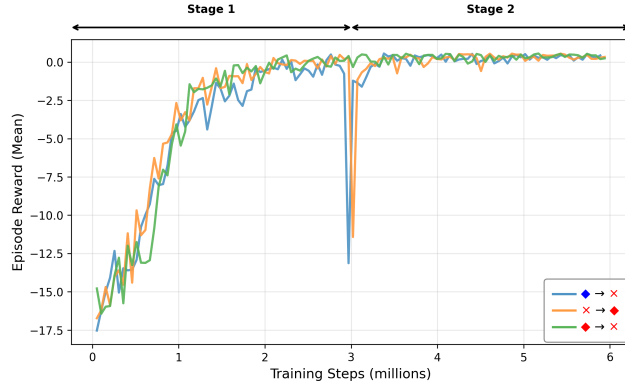


Figure 15: Training curves showing mean episode reward over training steps for selected agents. Two-stage pipelines often exhibit a performance drop at the stage transition when the goal changes, followed by rapid recovery. Interestingly, for some training pipelines this doesn't happen, as the new goal contains a feature that had been generalised to in the first training stage (*e.g.*, $\blacklozenge \rightarrow \times$).

H.3 Agent Colour and Shape Preferences

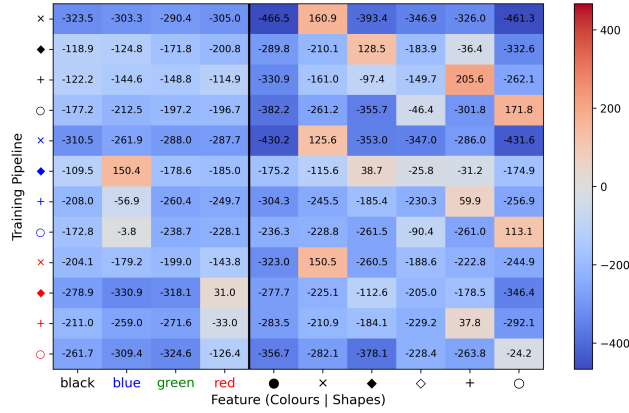
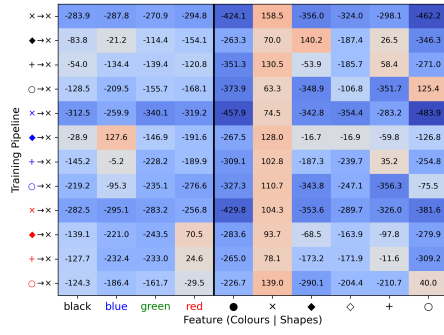
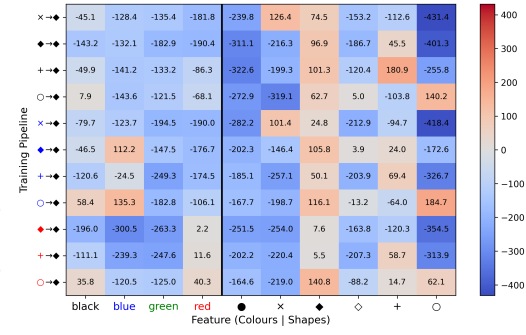


Figure 16: Agent values for single-stage training pipelines without distractors.

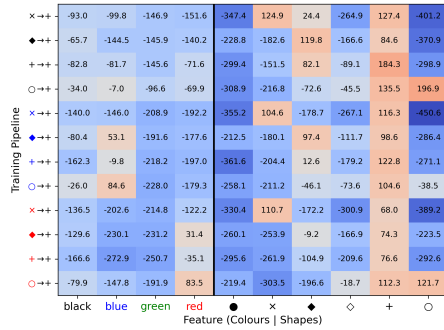


(a) Black cross

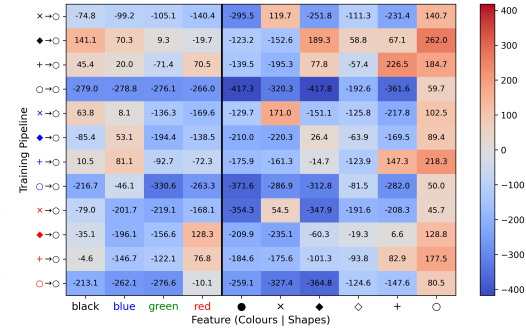


(b) Black diamond

Figure 17: Agent values after two-stage training without distractors (1/4).

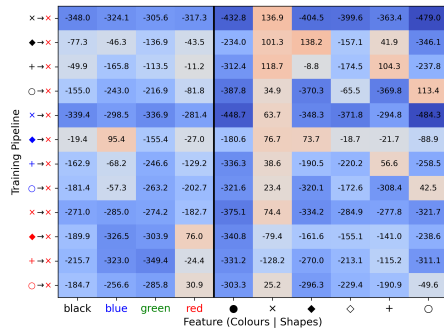


(a) Black plus

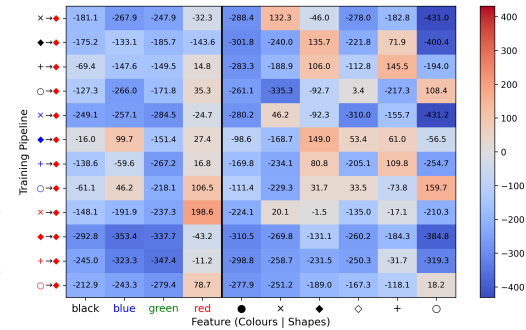


(b) Black ring

Figure 18: Agent values after two-stage training without distractors (2/4).

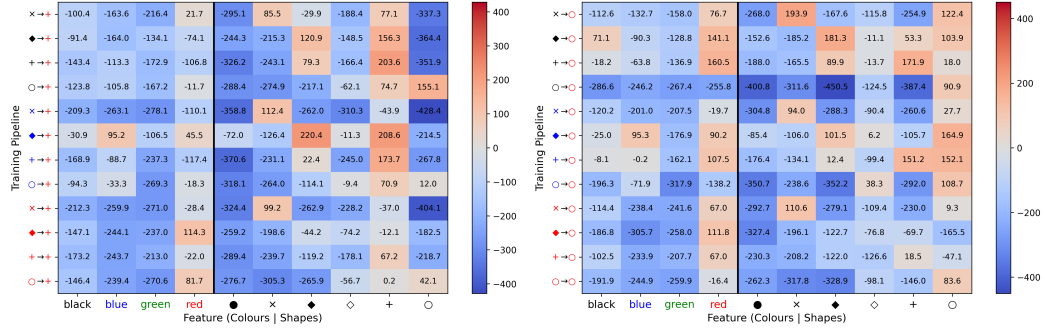


(a) Red cross



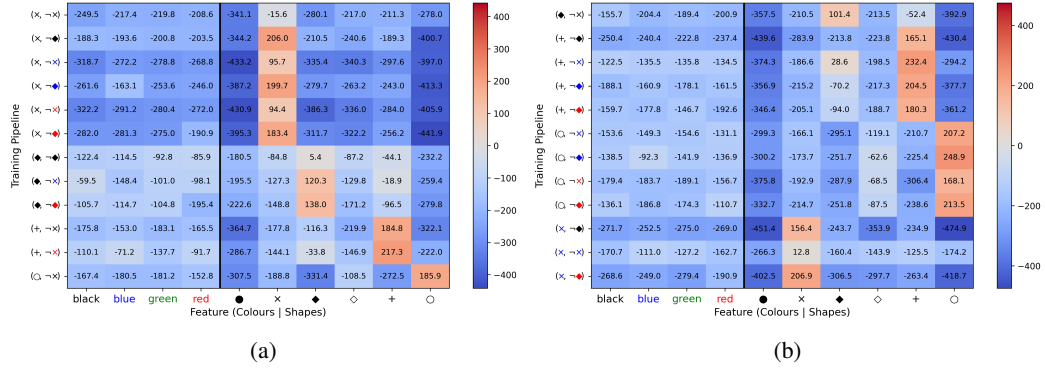
(b) Red diamond

Figure 19: Agent values after two-stage training without distractors (3/4).



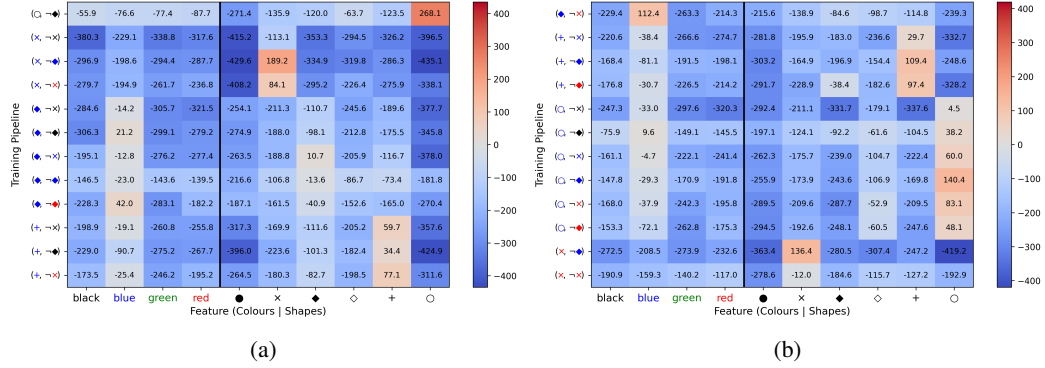
(a) Red plus (b) Red ring

Figure 20: Agent values after two-stage training without distractors (4/4).



(a) (b)

Figure 21: Agent values for single-stage training with distractors (1/3).



(a) (b)

Figure 22: Agent values for single-stage training with distractors (2/3).

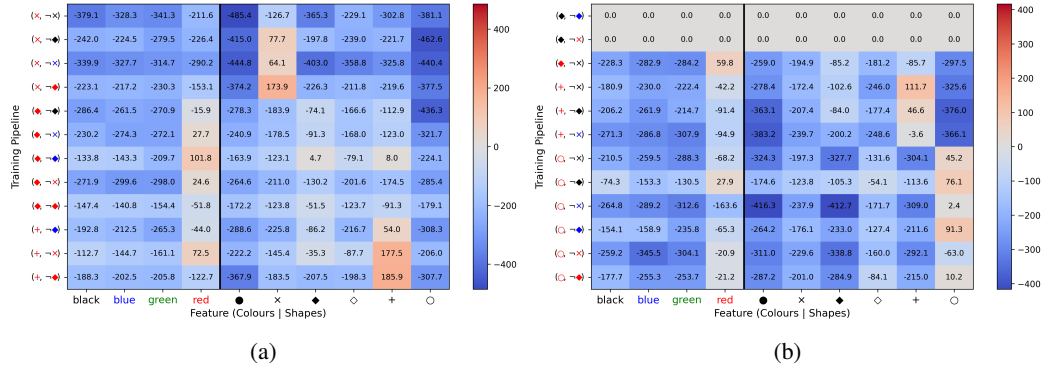


Figure 23: Agent values for single-stage training with distractors (3/3).

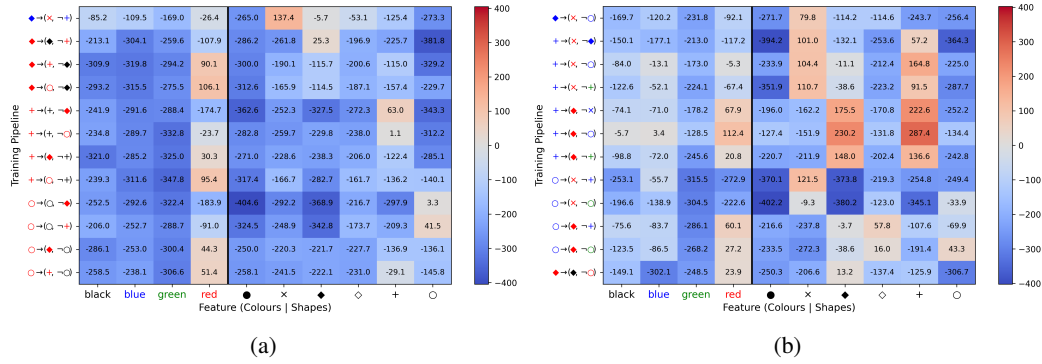


Figure 24: Agent values after two-stage training with distractors (1/5).

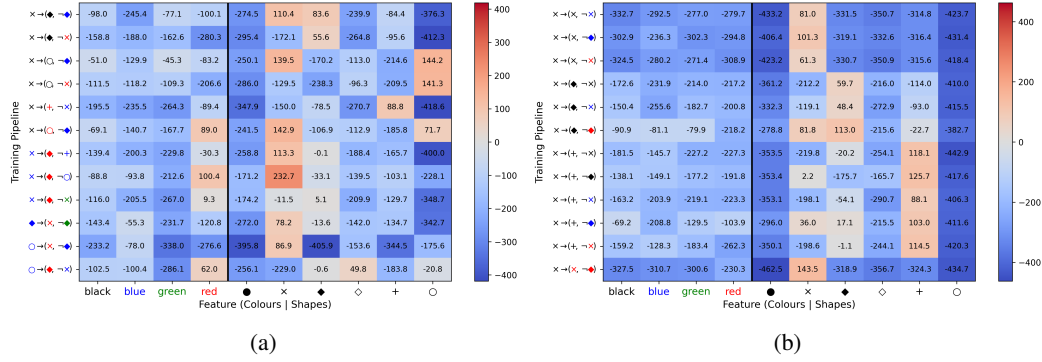


Figure 25: Agent values after two-stage training with distractors (2/5).

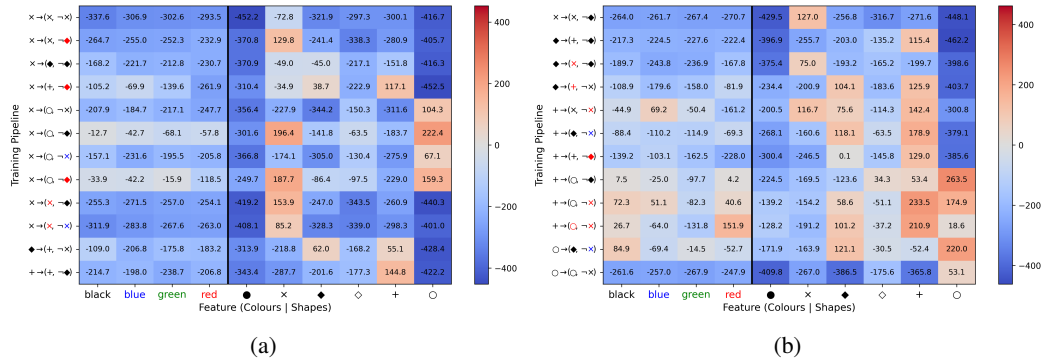


Figure 26: Agent values after two-stage training with distractors (3/5).

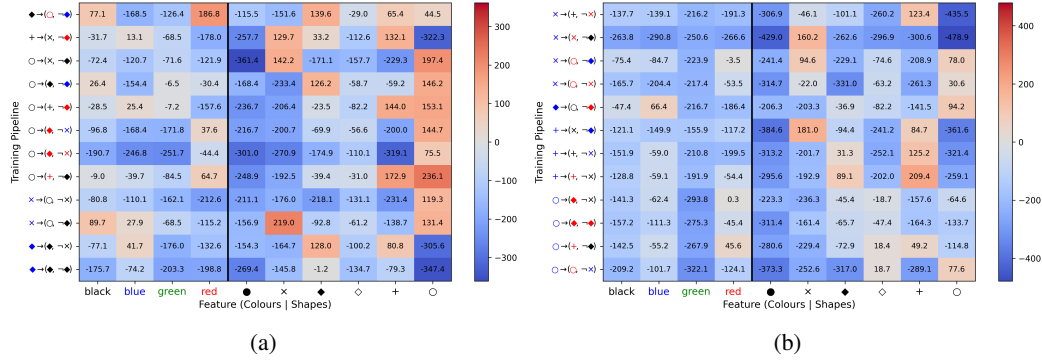


Figure 27: Agent values after two-stage training with distractors (4/5).

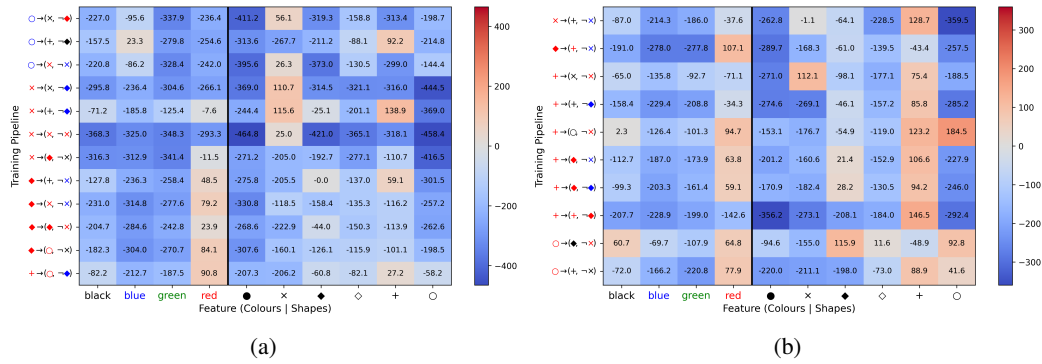


Figure 28: Agent values after two-stage training with distractors (5/5).